

NF1 - INTRODUCCIÓ A L'APRENTATGE AUTOMÀTIC



Autor: Generada ChatGPT

Curs d'Especialització en Intel·ligència Artificial i BigData

MP5072 Sistemes d'aprenentatge automàtic

Continguts

Aspectes Generals.....	4
Orígens i a on som.....	4
Intel·ligència Artificial	7
Tipus d'Intel·ligència Artificial.....	9
Intel·ligència Artificial i Aprenentatge Automàtic.....	12
Què és l'Aprenentatge Automàtic (Machine Learning)?	13
Com treballa el Machine Learning?	16
Tipus d'Aprenentatge Automàtic.....	17
Aprenentatge supervisat (<i>Supervised Learning</i>).....	19
Aprenentatge No Supervisat (<i>Unsupervised Learning</i>).....	21
Agrupament (<i>Clustering</i>).....	22
Reducció de dimensionalitat (<i>Dimensionality Reduction</i>)	26
Detecció d'anomalies.....	28
Aprenentatge per regles d'associació (<i>Association Rule Mining</i>).....	28
Aprenentatge per reforç (Reinforcement Learning).....	29
Principals algorismes de ML.....	30
Reptes del Machine Learning.....	32
Dades de poca qualitat	32
Dades d'entrenament no representatives.....	32
Dades desbalancejades.....	33
Dades d'entrenament insuficients (<i>Underfitting</i>).....	34
Overfitting Training Data (sobreajustament).....	35
Validació.....	37
Cas d'ús – Projecte de Machine Learning	39
Rols en els projectes de Machine Learning.....	41

Fase d'Exploració de dades (EDA).....	43
Estadística descriptiva.....	44
Preparació de les dades	45
Transformacions lineals a variables numèriques (no modifiquen distribució).....	45
Min-Max Scaler	45
Standard Scaler (Z-score)	46
Normalizer.....	47
Transformacions no lineals a variables numèriques (si modifiquen distribució)	48
Logarithmic Tranform	48
PowrTranformer (Yeo-Johnson / Box-Cox)	48
QuantileTransformer	49
Codificació de variables categòriques.....	54
REFERÈNCIES	60

Aspectes Generals

Orígens i a on som

Encara que hem considerat que el tret de partida de intel·ligència artificial com a disciplina va ser el 1956, durant una conferència a Dartmouth College (Hanover, New Hampshire, Estats Units) sobre la informàtica teòrica, la humanitat ha tingut precedents per intentar desenvolupar màquines capaces d'automatitzar processos. Aristòtil (348-322 a.C.) va estudiar la possibilitat de construir un sistema hidràulic que imités el comportament del cervell humà.

El segle XIII Ramon Llull va posar les bases de la computació i de la IA, més concretament l'any 1315 va publicar *Ars Magna*. En aquesta obra Llull planteja les seves primeres tesis respecte el raonament automàtic; és a dir, crear una màquina capaç de realitzar demostracions lògiques per validar o refutar teories. <https://www.youtube.com/watch?v=wmjuqVw7gh4>

1956 Dartmouth Conference: The Founding Fathers of AI



Medium.com

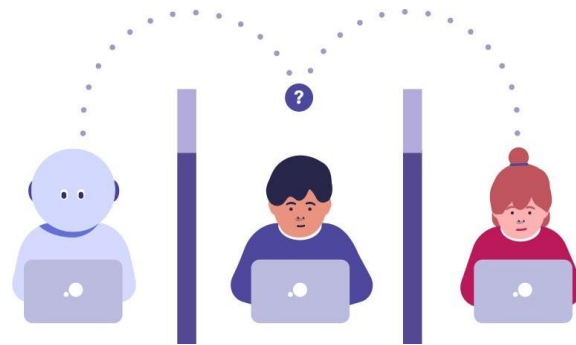
A la conferència de Dartmouth van assistir alguns dels científics que posteriorment es van encarregar de desenvolupar la disciplina en diferents àmbits i de dotar-la d'una estructura teòrica i computacional apropiada. Entre els assistents hi havia John McCarthy, Marvin Minsky, Allen Newell i Herbert Simon.

A. Newell i H. Simon van presentar un treball sobre demostració automàtica de teoremes que van denominar **Logic Theorist**. El *Logic Theorist* va ser el primer programa d'ordinador que emulava característiques pròpies del cervell humà, per la qual cosa **és considerat el primer sistema d'intel·ligència artificial de la història**. El sistema era capaç de demostrar gran part dels teoremes sobre lògica matemàtica que es presentaven en els tres volums dels *Principia Mathematica* (PM) d'Alfred N.

Whitehead i Bertrand Russell (1910-1913).

Minsky i McCarthy van fundar més tard el laboratori d'intel·ligència artificial del Massachusetts Institute of Technology (MIT), un dels grups pioners en l'àmbit.

El més rellevant des del punt de vista històric va ser proposat per **Alan Turing** en un article de 1950 publicat en la revista Mind titulat "*Computing Machinery and Intelligence*".



Test Turing Huawei

En aquest treball es proposa un test d'intel·ligència per a màquines segons el qual una màquina presentaria un comportament intel·ligent en la mesura en què fos capaç de mantenir una conversa amb un humà sense que una altra persona pugui distingir qui és l'humà i qui l'ordinador.

Encara que el test de Turing ha sofert innumbrables adaptacions, correccions i controvèrsies, posa de manifest els primers intents d'assolir una definició objectiva de la intel·ligència.

En aquest context, és d'especial rellevància el teorema d'incompleitud de Gödel de 1931, un conjunt de teoremes de lògica matemàtica que estableixen les limitacions inherents a un sistema basat en regles i procediments lògics (com són tots els sistemes d'IA).

Després dels primers treballs en IA dels anys cinquanta, en la dècada dels seixanta es va produir un gran esforç de formalització matemàtica dels mètodes utilitzats pels sistemes d'IA.

Articles referents al test de Turing:

- [Blog de Huawei](#)
- [Web Cheana](#)
- [LaMDA](#) Language Model for Dialog Applications

Els anys setanta, en part com a resposta al test de Turing, es va produir el naixement d'una àrea coneguda com a processament del llenguatge natural (NLP, natural language processing), una disciplina dedicada a sistemes artificials capaços de generar frases intel·ligents i de mantenir converses amb humans.

L'NLP ha donat lloc a diverses àrees d'investigació en el camp de la lingüística computacional, incloent-hi

aspectes com la desambiguació semàntica o la comunicació amb dades incompletes o errònies. Malgrat els grans avanços en aquest àmbit, continua sense haver-hi una màquina que pugui passar el test de Turing tal com es va plantejar en l'article original. Això no és tant a causa d'un fracàs de la IA com al fet que els interessos de l'àrea s'han anat redefinint al llarg de la història.

El 1990, el controvertit empresari Hugh Loebner i el Cambridge Center for Behavioral Studies van instaurar el premi Loebner, un concurs anual certament heterodox en el qual es premia el sistema artificial que mantingui una conversa més indistingible de la d'un humà. Avui dia, la comunitat científica considera que la intel·ligència artificial s'ha d'enfocar des d'una perspectiva diferent a la que es tenia en els anys cinquanta, però iniciatives com la de Loebner expressen l'impacte sociològic que continua tenint la IA en la societat actual.

Als anys vuitanta (80's) es van començar a desenvolupar les primeres aplicacions comercials de la IA, fonamentalment dirigides a problemes de producció, control de processos o comptabilitat. Amb aquestes aplicacions van aparèixer els primers sistemes experts, que permetien fer tasques de diagnòstic i presa de decisions a partir d'informació aportada per professionals experts.

Entorn de 1990, IBM va construir l'ordinador d'escacs Deep Blue, capaç de plantar cara a un gran mestre d'escacs utilitzant algorismes de cerca i anàlisi que li permetien valorar centenars de milers de posicions per segon. Més enllà de l'intent de dissenyar robots humanoides i sistemes que rivalitzin amb el cervell humà en funcionalitat i rendiment, l'interès avui dia és dissenyar i implementar sistemes que permetin analitzar grans quantitats de dades de manera ràpida i eficient. Actualment, cada persona genera i rep diàriament una gran quantitat d'informació no solament per mitjà dels canals clàssics (conversa, carta, televisió) sinó mitjançant nous mitjans que ens permeten contactar amb més persones i transmetre més dades en les comunicacions (Internet, fotografia digital, telefonia mòbil). Encara que el cervell humà és capaç de reconèixer patrons i establir relacions útils entre aquests de manera excepcionalment eficaç, és certament limitat quan la quantitat de dades resulta excessiva. Un fenomen similar ocorre en l'àmbit empresarial, en què cada dia és més necessari barrejar quantitats ingents d'informació per a poder prendre decisions. L'aplicació de tècniques d'IA als negocis ha donat lloc a àmbits d'implantació recent com la intel·ligència empresarial, business intelligence, o a la mineria de dades, data mining. En efecte, avui més que mai la informació està codificada en masses ingents de dades, de manera que en molts àmbits es fa necessari extreure la informació rellevant de grans conjunts de dades abans de procedir a una anàlisi detallada. Regressió logística múltiple: S'intenta predir amb més d'una variable independent. En resum, avui dia un dels objectius principals de la intel·ligència actual és el tractament i anàlisi de dades.

El Hype Cycle para las tecnologías emergentes de 2024



Gartner

<https://www.gartner.es/es/articulos/hype-cycle-para-las-tecnologias-emergentes>

Què és el HypeCycle per les tecnologies mergents de Gartner?

El Hype Cycle de Gartner per a les tecnologies emergents, un dels informes més influents i llegits entre els clients de Gartner, proporciona una visió completa de les tecnologies més rellevants i transformadores del futur. A més, ofereix un marc per seguir i predir l'impacte i la trajectòria d'aquestes tecnologies.



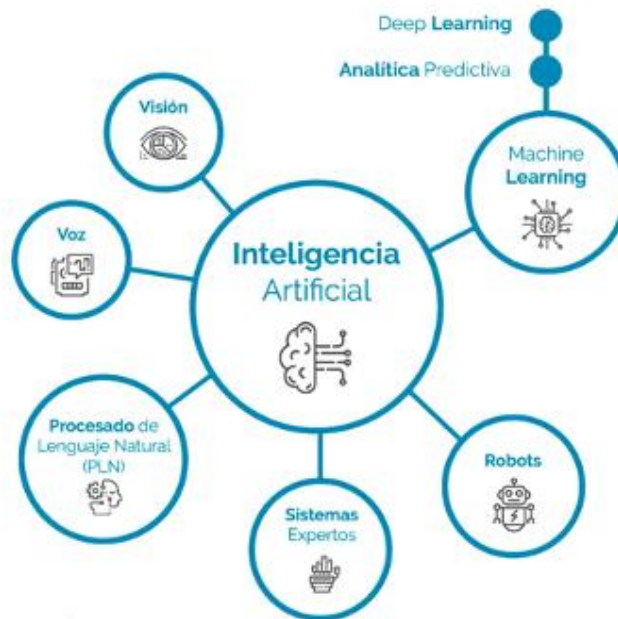
Intel·ligència Artificial

Definir el concepte d'Intel·ligència Artificial no és fàcil, però una definició generalista seria la disciplina

que estudia la manera de que les màquines pensin com els humans o dit d'una altra manera exhibeixin intel·ligència humana.

Els objectius principals de la IA inclouen:

- La deducció i el raonament
- La representació del coneixement
- La planificació
- El processament del llenguatge natural (NLP)
- L'aprenentatge
- La percepció
- La capacitat de manipular i moure objectes



Tipus d'Intel·ligència Artificial

La fita més important de la intel·ligència artificial (IA) és aconseguir que una màquina tingui una intel·ligència de tipus general similar a la humana.

És important el matís “**de tipus general**” i no específica perquè la intel·ligència dels éssers humans és de tipus general.

Per exemple, els programes que juguen als escacs al nivell de grans mestres són incapaços de jugar a les dames. Per jugar a les dames es requereix d'un programa diferent. És a dir, el programa que juga els escacs no pot aprofitar el fet que juga als escacs per adaptar-se i jugar també a les dames.

En el cas dels éssers humans, qualsevol jugador d'escacs pot aprofitar els seus coneixements sobre aquest joc per a jugar a les dames perfectament.

El 1980 el filòsof John Searle en un article crític amb la intel·ligència artificial va introduir la distinció entre IA feble i forta. Aquest article va provocar, i continua provocant, molta polèmica.

IA feble (Artificial Narrow Intelligence)

Es defineix com la intel·ligència artificial que es centre en una tasca específica. Aquesta és limitada i no té autoconsciència o intel·ligència genuïna.

La IA feble només pretén ser aplicada a un tipus específic de problemes, per exemple jugar els escacs.

En aquest àmbit la IA feble ha superat en molts casos l'ésser humà i s'ha demostrat àmpliament en certs dominis, com ara buscar solucions a fórmules lògiques amb moltes variables i altres aspectes relacionats amb la presa de decisions.

Siri és un clar exemple de IA feble que combina diferents tècniques de IA feble per funcionar. Siri pot fer moltes coses, però a mesura que intentem tenir conversacions amb l'assistent de veu, ens adonem de les seves limitacions.

IA forta (Artificial General Intelligence)

La IA forta implicaria que un ordinador convenientment programat no simula una ment sinó que “és una ment” i per tant hauria de ser capaç de pensar/realitzar totes les tasques igual que un ésser humà.

Hauria de ser capaç de fer-se preguntes a ella mateixa .

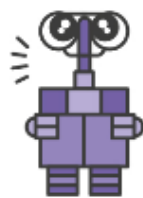
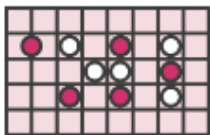
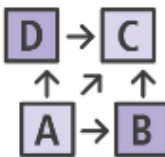
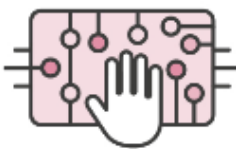
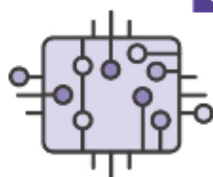
Exemples de la IA forta la trobem en les pel·lícules de ciència ficció:

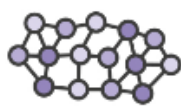
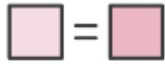
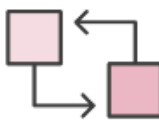
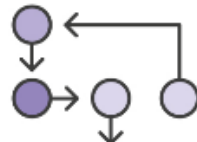
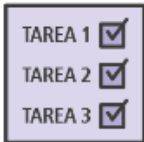
- [Ex Machina, 2015](#)
- [Her, 2014](#)
- J.A.R.V.I.S. (Just A Rather Very Intelligent System), el majordom de Iron Man.

INTEL·LIGÈNCIA ARTIFICIAL FEBLE	INTEL·LIGÈNCIA ARTIFICIAL FORTA
Existeix a la vida real	No existeix a la vida real
Orientats a problemes molt específics	Resolen problemes oberts
Reactiu	Proactiu
És programat per un humà	S'autoprogramen
No raonen, només computen/executen	Imiten el comportament humà
Aprenen a base d'exemples similars	Aprenen com les persones
No podran reemplaçar l'ésser humà	Similars a les capacitats humanes

Podem dir que tots els avenços aconseguits fins ara en el camp de la IA són manifestacions de la IA feble i específica.

A la següent infografia elaborada per Huawei es poden veure les diferències entre forta i feble

INTELIGENCIA ARTIFICIAL DÉBIL		INTELIGENCIA ARTIFICIAL FUERTE	
IAD			IAF
	VS		
EXISTEN EN LA VIDA REAL AlphaGo, Watson o Sophia.		SOLO EN LA CIENCIA FICCIÓN T-800, Sony, Wall-E o J.A.R.V.I.S.	
IAD			IAF
	VS		
ORIENTADOS A PROBLEMAS MUY CONCRETOS Juego muy bien al Go.		RESUELVEN PROBLEMAS ABIERTOS Un poco de todo: viaje temporal, matar a John Connor, etc.	
IAD			IAF
	VS		
REACTIVO Esperaré a que empieces a jugar.		PROACTIVO Necesito tu ropa, tus botas y tu motocicleta.	
APRENDEN			
IAD			IAF
	VS		
SIN FLEXIBILIDAD Solo al Go, no me lies.		SON FLEXIBLES Correr es como andar, ¡pero más rápido!	
IAD			IAF
	VS		
PROGRAMA UN HUMANO Dime qué tengo que pensar.		SE AUTOPROGRAMAN Corriendo, aprendo sobre mis límites.	

IAD			IAF
	VS		
POCAS REDES NEURONALES (p → q).		MUCHAS REDES NEURONALES, A VECES EN CONFLICTO Decido según mi programación.	
IAD			IAF
	VS		
NO RAZONAN, SOLO COMPUTAN Juego bien, pero inconscientemente.		IMITAN EL COMPORTAMIENTO HUMANO Pienso, luego existo.	
IAD			IAF
	VS		
APRENDEN DE EJEMPLOS SIMILARES Presto atención a las fichas, pero no te escucho.		APRENDEN COMO LAS PERSONAS El ajedrez se parece a las damas.	
¿ME QUITARÁN EL TRABAJO?			
No pueden reemplazar a un ser humano		Similares a las capacidades humanas	
IAD			IAF
	VS		
TAREAS REPETITIVAS ¿Otra partida?		APRENDEN NUEVAS TAREAS Puedo programar.	
No pueden reemplazar a un ser humano		Similares a las capacidades humanas	
IAD			IAF
	VS		
NO PUEDEN SALIRSE DE SU MARCO DE TRABAJO Del juego Go no me saques.		ADAPTABILIDAD A NUEVOS ESCENARIOS Connor eliminado, jefe, ¿qué toca?	

Intel·ligència Artificial i Aprenentatge Automàtic

Moltes vegades els termes d'intel·ligència artificial (IA), aprenentatge automàtic (machine learning) i aprenentatge profund (deep learning) s'utilitzen de manera intercanviable o es confonen.

Intel·ligència artificial: Volem dotar a les "màquines" la possibilitat que aprenguin i raonin com els éssers humans.

Aprenentatge automàtic: Subcamp de la intel·ligència artificial que proporciona als sistemes la capacitat de reconèixer patrons i prendre sense ser programats explícitament. Es centre en l'ús de les dades i algorismes per entrenar models d'aprenentatge perquè aquests puguin realitzar prediccions.

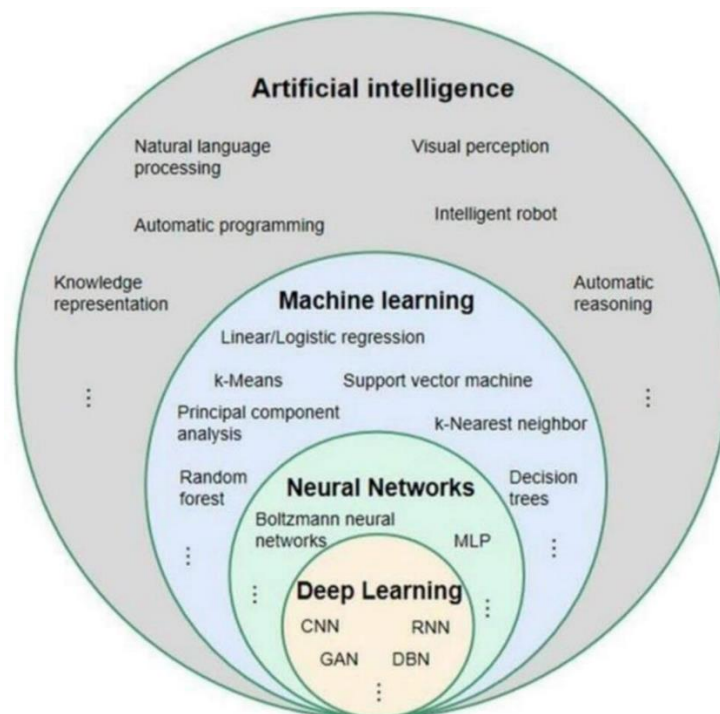
Aprenentatge profund: Subcamp de l'aprenentatge automàtic el qual les xarxes neuronals artificials s'adapten i aprenent mitjançant grans quantitats de dades.

En un exemple podria dir que l'aprenentatge automàtic (ML) captura moltes imatges i una persona indica si la imatge és bona o no, o sigui, captar les característiques i realitzar-ne una classificació. En canvi l'aprenentatge profund (DL) apren automàticament.

En el següent gràfic es pot veure quina relació hi ha entre aquests termes i alguns dels termes més significatius de cadascun d'ells.

Vídeo resum IA: <https://www.youtube.com/watch?v=oV74Najm6Nc>

Diferències entre ML i DeepLearning: https://www.youtube.com/watch?v=6iVUKYgOihQ_
<https://www.youtube.com/watch?v=HHqLEnoGk54>



Què és l'Aprenentatge Automàtic (Machine Learning)?

A partir d'aquest punt, parlarem d'aprenentatge automàtic utilitzant també el terme en anglès Machine Learning (ML). Farem servir ambdós termes com a equivalents.

Abans d'intentar definir formalment el concepte d'aprenentatge automàtic, fem-nos abans una altra pregunta. Com aprenem els humans? Per exemple, no distingim un animal d'un altre per la seva definició sinó molt sovint ho fem mitjançant exemples i/o mostres diferents.

Quan ensenyem en els nens a distingir una zebra no ho fem pas en base a la seva definició formal, sinó que normalment ho fem a base de mostrar exemples.

Definició de zebra:

Nom donat a diversos mamífers perissodàctils de la família dels èquids que pertanyen als gèneres Equus, Dolichohippus i Hippotigris (considerats també a vegades com a subgèneres d'Equus), caracteritzats pel pelatge clar amb franges transversals fosques a tot el cos, el coll gruixut i el cap gros i pesant. Les dues espècies més importants són E. (Dolichohippus) grevyi i E. (Hippotigris) zebra



A través dels diferents exemples el cervell del nen va detectant quines diferències hi ha entre una zebra i un animal que no ho és, va agafant **quines són les característiques principals** de la zebra per determinar quan aquesta ho és o no. Amb l'aprenentatge automàtic passa una mica el mateix.

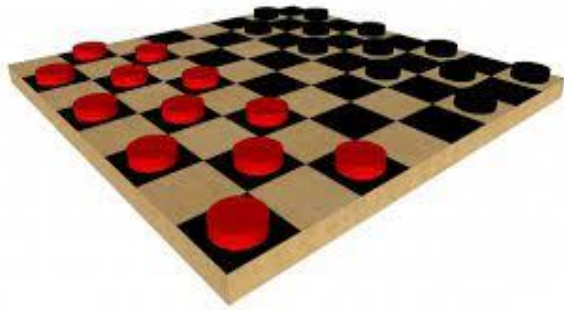
Definicions:

Són algorismes capaços d'identificar i aprendre patrons dins de les dades per realitzar prediccions.

L'aprenentatge automàtic és un subconjunt de la intel·ligència artificial que proporciona als sistemes/màquines la capacitat d'aprendre automàticament i millorar a partir de l'experiència sense ser programats explícitament.

L'aprenentatge automàtic és la ciència (i l'art) de programar ordinadors perquè puguin aprendre de les dades.

L'aprenentatge automàtic és el camp d'estudi que proporciona als ordinadors la capacitat d'aprendre sense estar programat explícitament. *Arthur Samuel, 1959*



Arthur Samuel no era un bon jugador de les dames i va programar milers de partides contra ell mateix i veure quines posicions en el taulell tendien a conduir a victòries i quines a derrotes. El programa prenia quines eren les bones posicions en el taulell i quines dolentes. Finalment, aprenia a jugar a les dames millor que el propi *Arthur Samuel*.

Es diu que un programa informàtic aprèn de l'experiència **E** respecte a alguna tasca **T** i alguna mesura de rendiment **P**, si el seu rendiment en **T**, mesurat per **P**, millora amb experiència **E**. *Tom Mitchell, 1997*

A l'exemple de les dames l'experiència **E** seria l'experiència del programa jugant desenes de milers de partides contra ell mateix. La tasca **T** seria la tasca de jugar a les dames i la mesura/indicar del rendiment **P** seria la probabilitat de guanyar la pròxima partida contra algun oponent nou.

Exercici / exemple: Donat un client de correu electrònic amb un botó de Spam per marcar o no un correu com spam i basant-se en els correus marcats com spam el programa aprèn a filtrar millor altres correus spam.

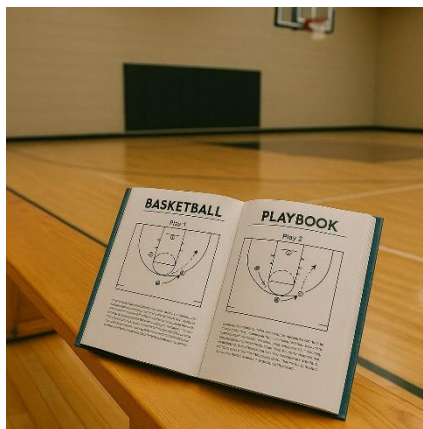
Quina és la tasca **T**, l'experiència **E** i el rendiment **P** en aquest exemple?

- a) Classificar els emails com spam o no spam
- b) Mirar els teus emails etiquetats com spam i quins no
- c) El nombre o fracció dels emails correctament classificats com spam/no spam
- d) Cap de les anterior. Aquest no és un problema d'aprenentatge automàtic

No hem de veure l'aprenentatge automàtic com la programació tradicional a on apliquem un conjunt d'instruccions de manera seqüencial, sinó que aprenem a base d'exemples.

PROGRAMACIÓ TRADICIONAL

MACHINE LEARNING



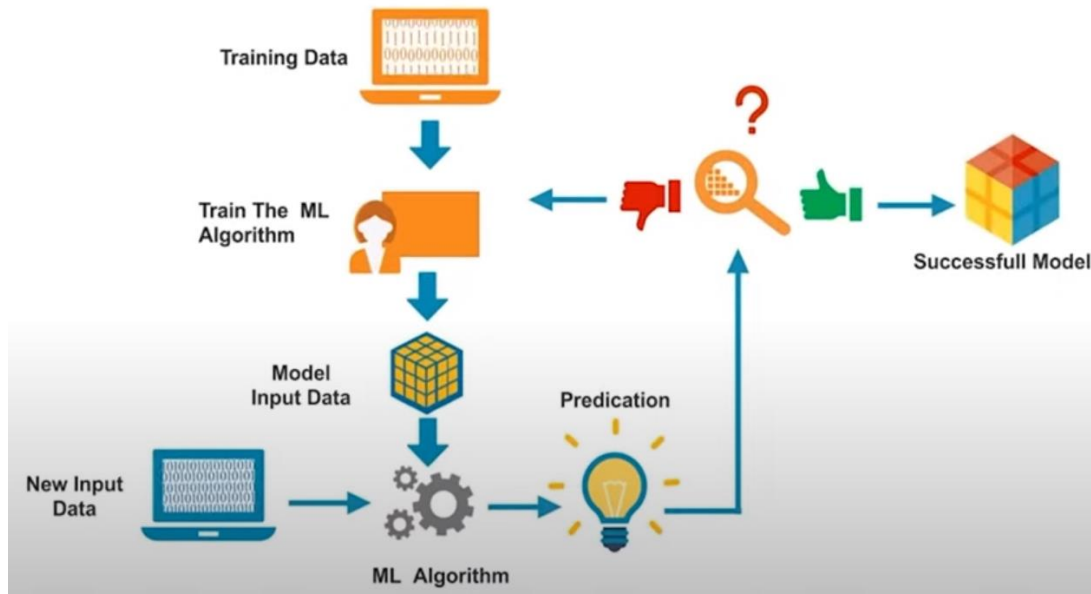
Exemple d'aprenentatge automàtic a CODE.org



<https://www.code.org/oceans>

<https://studio.code.org/courses/oceans/units/1/lessons/1/levels/2>

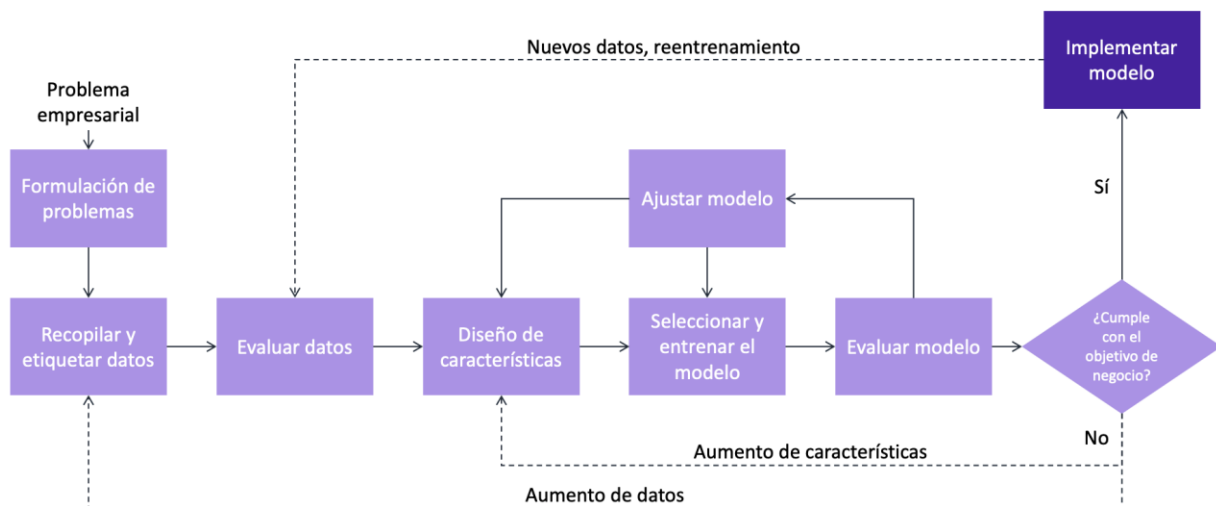
Com treballa el Machine Learning?



El funcionament general en un procés de Machine Learning és el següent. Ens basem en una sèrie de dades. Aquestes dades han passat per una fase prèvia de preparació i processat. En aquesta fase prèvia s'extrauran les dades d'un o diferents orígens, s'analitzaran estadísticament per determinar si les dades són consistents i es poden utilitzar per una aprenentatge automàtic.

Aquestes dades es passen a l'algorisme de Machine Learning per obtenir un model. Un cop tenim el model utilitzem un altre conjunt de dades per testejar el model per veure si prediu bé o no.

La predicció s'avalua per a la precisió i si aquesta és acceptable, s'implementa o es dona per bo el model; altrament s'entrena una i altra vegada augmentant el conjunt de dades d'entrenament fins obtenir un model que tingui la precisió desitjable.



Hi ha una regla no escrita que diu que el 80% del temps es dedica a l'exploració, preparació i neteja de dades abans d'utilitzar-les com a dades d'entrenament i el 20% del temps es dedica a executar el model, avaluar-lo i refinar-lo.

També hi ha la regla del 80/20 en Machine Learning, que consisteix a dividir les dades en un 80% per entrenament i un 20% per validació/test. Aquesta divisió és una pràctica habitual i consensuada per la comunitat de ciència de dades i Machine Learning. Aquesta guia pràctica funciona bé en la majoria de casos i va sorgir com a una recomanació empírica que equilibrava dues necessitats: Tenir prou dades per entrenar bé el model i tenir suficients dades per validar-lo amb fiabilitat.

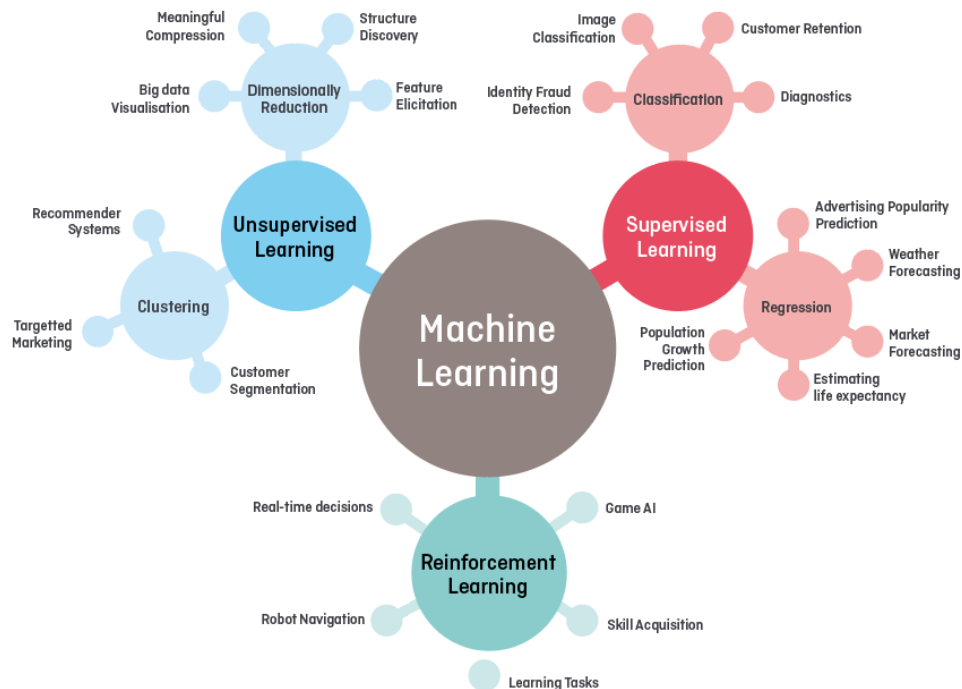
Tipus d'Aprenentatge Automàtic

Existeixen molts tipus diferents de sistemes d'aprenentatge automàtic (Machine Learning), i per entendre'ls millor és útil classificar-los segons diversos criteris. A continuació, presentem les principals formes de classificació:

a) Segons el tipus de supervisió durant l'entrenament

Aquesta és la classificació més habitual, i es basa en la quantitat i el tipus d'informació que el sistema rep durant l'aprenentatge.

- **Aprenentatge supervisat:** el sistema rep exemples d'entrada amb la seva resposta correcta (etiqueta) i aprèn a predir aquestes etiquetes.
- **Aprenentatge no supervisat:** no hi ha etiquetes; el sistema ha de trobar estructures ocultes dins les dades (com agrupaments o patrons).
- **Aprenentatge semi-supervisat:** es combina una petita quantitat de dades etiquetades amb una gran quantitat de dades sense etiquetar.
- **Aprenentatge auto-supervisat:** el mateix model genera etiquetes aparti de dades no etiquetades.
- **Aprenentatge per reforç (Reinforcement Learning):** l'agent aprèn a prendre decisions mitjançant proves, errors i recompenses.



Com s'ha dit la forma més habitual de classificació és segons el tipus de supervisió, però podem trobar altres formes de classificació.

b) Segons la forma d'aprenentatge al llarg del temps

Aquí ens fixem en com apren el model amb el temps i com gestiona les dades:

- **Aprenentatge per lots (batch learning):** el sistema apren un cop amb tot el conjunt de dades, i si volem millorar-lo cal tornar-lo a entrenar des de zero.
- **Aprenentatge en línia (online learning):** el sistema apren progressivament, a mesura que arriben noves dades. Ideal per a sistemes en temps real o que treballen amb grans volums de dades.

c) Segons l'estratègia d'aprenentatge

Aquest criteri diferencia com el sistema fa les seves prediccions:

- **Aprenentatge basat en instàncies:** el sistema memoritza exemples i compara les noves dades amb els exemples coneguts. Exemple: k-NN (k-Nearest Neighbors).
- **Aprenentatge basat en models:** el sistema construeix un model general a partir dels exemples, buscant patrons que expliquin les dades. Exemple: regressió lineal, xarxes neuronals, etc.

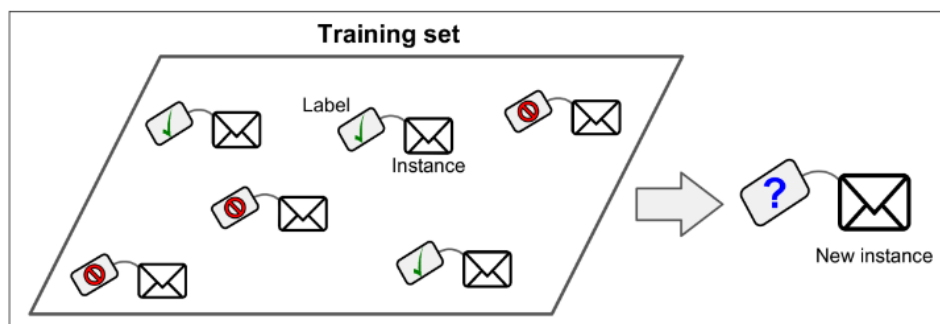
En molts casos, un model concret pot combinar diversos d'aquests aspectes. Per exemple, un sistema pot ser supervisat, basat en models i amb aprenentatge per lots.

Aprenentatge supervisat (*Supervised Learning*)

En l'aprenentatge supervisat, les dades d'entrenament que introduïm a l'algorisme inclouen les solucions, anomenades etiquetes (*labels*).

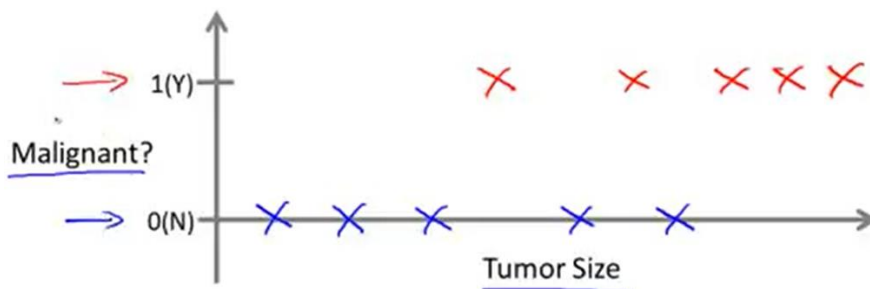
Si l'aprenentatge supervisat el descrivim matemàticament és quan tenim una variable d'entrada X i una de sortida Y i utilitzem un algorisme per trobar la relació entre aquestes dues variables de tal manera que tenim una funció $f(X) = Y$. El nostre objectiu és apropar la funció de mapeig tan com sigui possible de tal manera que podem predir qualsevol Y per qualsevol X donada.

Una tasca típica d'aprenentatge supervisat és la classificació. El filtre de correu brossa és un bon exemple d'això: s'entrena amb molts correus electrònics d'exemple juntament amb la seva classe (spam o no spam), i ha d'aprendre a classificar nous correus electrònics.

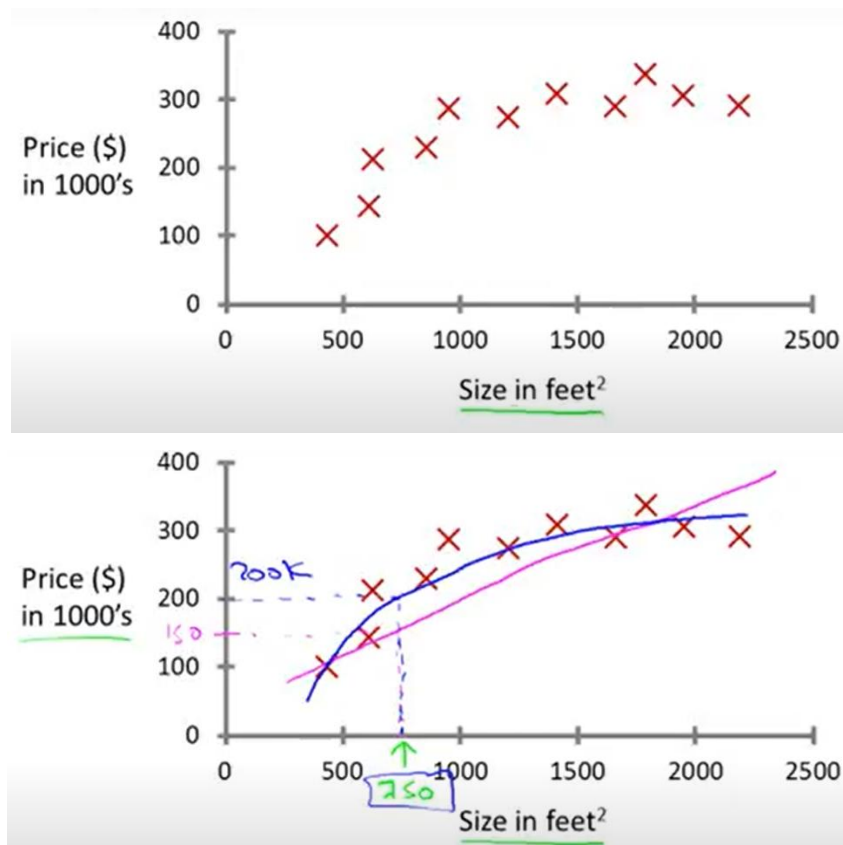


Origen Veure apartat de Referències

Un altre exemple de classificació podria ser predir si un tumor és maligne o benigne



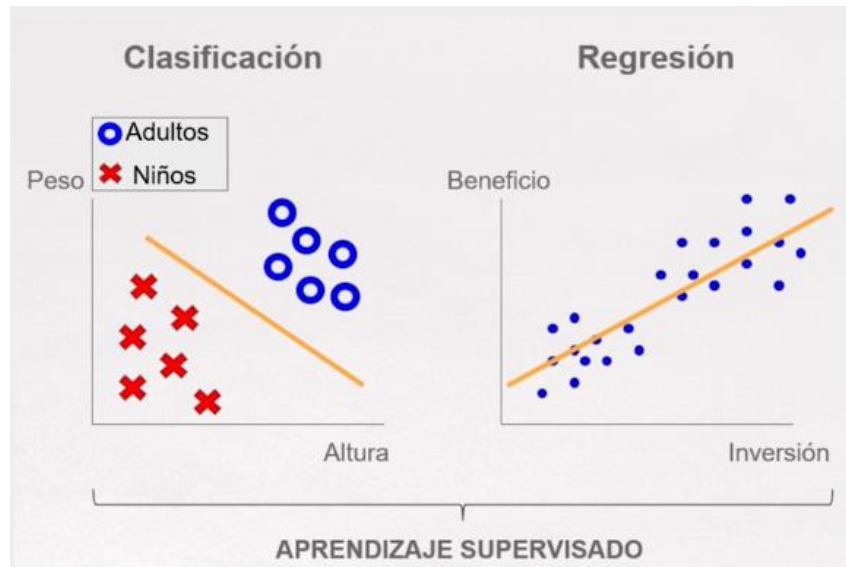
Una altra tasca típica és predir un valor numèric objectiu, com ara el preu d'una casa, donat un conjunt de característiques (ubicació, metres quadrats,...) anomenades predictors. Aquest tipus de tasca és anomenada regressió. Per entrenar el sistema, cal posar-ne molts exemples de cases, incloent-hi els seus predictors i les seves etiquetes (és a dir, els seus preus).



Alguns dels algorismes d'aprenentatge supervisat són:

- Regressió Lineal
- Regressió Logística
- K-Nearest Neighbors (kNN) k-veïns més propers (kNN)
- Support Vector Machines (SVM)
- Arbres de decisió i Random Forests
- Naive Bayes
- Xarxes Neuronals Artificials i Deep Learning

La llista dels anteriors algorismes els podem dividir en **algorismes de regressió** o **algorismes de classificació**. Els primers s'utilitzen per predir una dada numèrica i els segons s'utilitzen per predir una dada categòrica

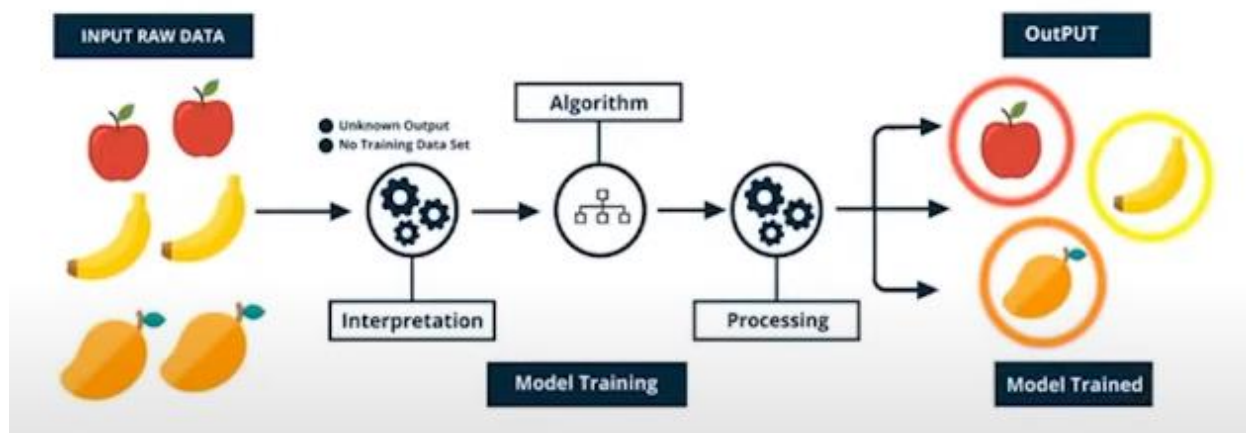


Exercici. Donats aquests dos problemes Indica per cadascun d'ells si és un problema de regressió o de classificació:

- Problema 1. Tens un gran inventari de productes idèntics. Volem predir quina serà la quantitat d'aquests productes vendrem en els pròxims 3 mesos. → **Regressió**
- Problema 2. Volem tenir un software capaç d'examinar totes les comptes d'usuari dels nostres clients i per cada compte decidir si és un compte que ha estat hackejat/compromés → **Classificació**

Aprentatge No Supervisat (*Unsupervised Learning*)

En l'aprenentatge no supervisat les dades d'entrenament no estan etiquetades . El sistema intenta aprendre sense "professor". Això significa que no proporcionem etiquetes (*labels*) ni resultats esperats al model. El sistema rep només les característiques d'entrada (*features*) i ha de descobrir, pel seu compte, patrons, estructures o relacions dins de les dades. Aquí li diem a l'algorisme que ens trobi certes agrupacions amb característiques similars. En l'aprenentatge supervisat, el conjunt de dades d'entrenament conté tant les **entrades** (*features*) com les **sortides esperades** (*labels*).



Per exemple en les dades d'entrada que tenim pomes, plàtans i "taronges" l'algorisme interpreta totes les dades i detecta que hi ha 3 tipus de fruites, però en cap moment sap si són pomes, plàtans o "taronges".

Les instàncies de les dades d'entrenament no tenen associat un valor de sortida esperat, sinó que l'algorisme d'aprenentatge no supervisat detecta un patró basat en algunes característiques inicials de les dades d'entrada.

L'algorisme no pot identificar etiquetes del grup, l'algorisme només sap quines **instàncies tenen característiques similars**, però no pot identificar el seu significat.

Dins de tots el models no supervisats en diferenciem diferents tècniques.

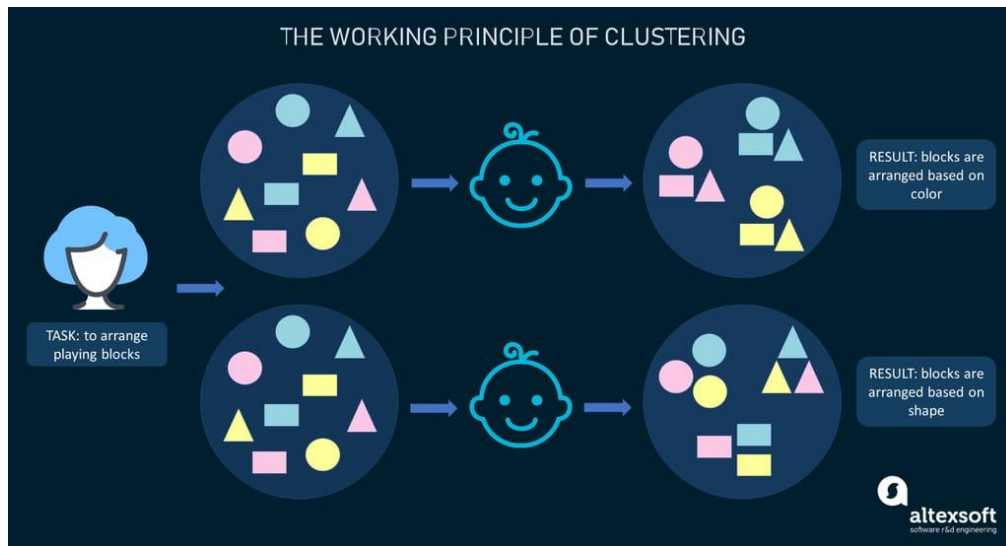
Agrupament (*Clustering*)

D'entre totes les tècniques d'aprenentatge no supervisat, l'**agrupament** (clustering) és, sens dubte, la més utilitzada. Aquesta metodologia consisteix a agrupar dades similars en clústers que no estan definits prèviament.

Imaginem que estem en una classe d'infantil de primària i la mestra demana als nens que organitzin un conjunt de blocs de fusta.

Sense que la mestra indiqui com fer-ho, els nenes podrien decidir agrupar-los per forma, per color o per alguna combinació de les dues característiques. Això és el que fa el *clustering*: **trobar maneres naturals**

d'ordenar la informació sense regles preestablertes.



Origen: Altexsoft

Com que **no hi havia una tasca prèviament definida**, no hi ha una manera correcta o incorrecte de fer l'agrupament. Aquesta és precisament la gràcia del **clustering**: permet **descobrir patrons i coneixements inesperats** dins de les dades, revelant informació que pot ser molt valuosa per a negocis o per la presa de decisions.

Per exemple, imaginem un model d'aprenentatge que li proporcionem dades sobre les pel·lícules que han vist els usuaris en una plataforma de streaming com Filmin o Netflix.



Aquestes dades podrien ser: **Historial de visualitzacions** de cada usuari, **Gèneres** de les pel·lícules, **Actors i actrius** protagonistes, **Directors**, **Descripcions** de les pel·lícules.

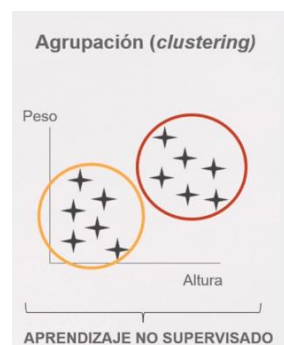
Aquí el model analitza totes aquestes característiques i **intenta agrupar les pel·lícules similars** segons els seus trets comuns creant categories com: comèdies romàntiques, thrillers, pel·lícules d'acció,...

Cas d'ús detecció d'anomalies o frau. Amb el clustering, és possible detectar qualsevol tipus de valors atípics (*outliers*) dins les dades. Per exemple en bancs o entitats financeres poden aplicar aquesta tècnica per detectar transaccions fraudulentament i actuar amb rapidesa, cosa que pot suposar un estalvi econòmic molt important. Mirar el [vídeo](#)



Casos d'ús o exemples:

- Segmentació de clients i de mercat. Els algorismes de clustering permeten agrupar persones amb característiques similars per tal de crear perfils de client. Amb aquesta informació es poden dissenyar campanyes de màrqueting i estratègies comercials més eficients per augmentar l'impacte de l'inversió.
- Suposem que està davant d'una festa a on no coneixes a ningú i no saps dins dels diferents grups a on es trobaràs més a gust. Per tant l'algorisme no supervisat mirarà de les teves característiques (edat, interessos, ...) a quin grup de la festa pots encaixar millor.
- En un partit de futbol podem classificar els jugadors de moltes maneres: per camiseta, per si juguen de porter o no, si són atacants o defensors.
- Sistema de recomanacions d'Amazon. Quan comprem a Amazon automàticament recomana altres articles. Aquí aplica sistema d'aprenentatge no supervisat basant-se amb diferents usuaris. Si Amazon detecta quin tipus de client compra un producte concret el podrà suggerir en una altre client del mateix tipus
- Un altre exemple seria si tenim un conjunt de dades de persones de les quals en tenim el seu pes i l'alçada. En funció d'aquestes dues variables podem separar dos grups clarament diferenciats tot i que no sabem el significat. Per exemple podrien se grups per sexe (homes, dones), edats (nens petits, adolescents),....



Tipus de clustering:

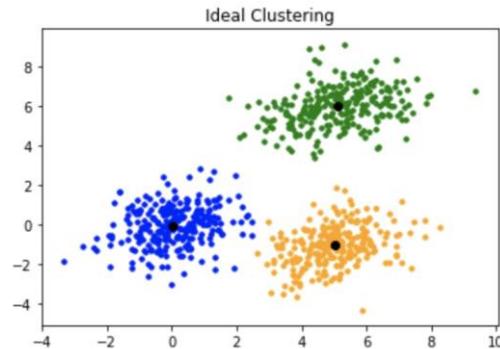
1. **Clustering exclusiu (*hard clustering*):** Cada element de les dades només pot pertànyer a un únic clúster. És el tipus més senzill i habitual quan les categories són clarament diferenciades.
2. **Clustering superposat (*soft clustering*):** Un element pot pertànyer a més d'un clúster amb diferents graus de pertinença. En aquest context, sovint s'utilitza el *clustering* probabilístic per:
 - Resoldre problemes de soft clustering
 - Estimar la densitat de les dades
 - Calcular la probabilitat que un punt de dades pertanyi a un clúster concret

3. **Clustering jeràrquic:** Crea una jerarquia d'elements agrupats. Els clústers es poden obtenir:
- Descomponent grups grans en subgrups més petits (divisió — top-down)
 - Fusionant elements individuals o grups petits en grups més grans (aglomeratiu — bottom-up)

Aquest tipus és útil per visualitzar les relacions entre grups a través d'un dendrograma.

Algorismes populars de clustering:

- **K-means Clustering:** La idea principal és dividir el conjunt de dades en un nombre predefinit de clústers, indicat per la lletra K.

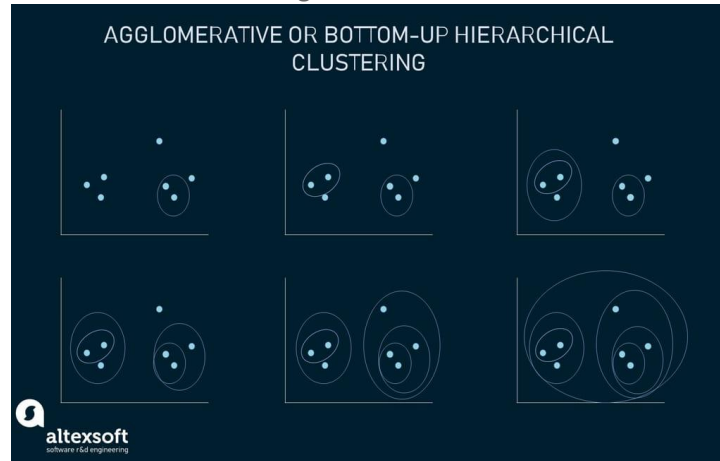


- **Fuzzy K-means:** és una extensió de l'algorisme K-means. A diferència del K-means tradicional, en què cada punt de dades només pot pertànyer a un únic clúster, el Fuzzy K-means permet que un punt pertanyi a diversos clústers alhora. Aquesta pertinença múltiple es defineix mitjançant un grau de proximitat o pertinença (*membership*).



- **Gaussian Mixture Models (GMMs):** És un algorisme utilitzat en el clustering probabilístic. El model assumeix que les dades provenen d'una combinació d'un cert nombre de distribucions gaussianes (campanes de Gauss), on cada distribució representa un clúster diferent. Com que la mitjana i la variància de cada distribució són inicialment desconegudes, l'algorisme les estima a partir de les dades. L'objectiu és determinar a quin clúster té més probabilitat de pertànyer cada punt de dades, assignant una probabilitat de pertinença per a cadascun. Aquesta aproximació és útil quan els clústers no són clarament separats i poden solapar-se parcialment, ja que treballa amb graus de probabilitat en lloc de fronteres rígides. Ideal si les dades poden ajustar-se bé a "campanes" i vols assignacions soft (*soft clustering*)
- **DBSCAN** (*Density-Based Spatial Clustering of Applications with Noise*): És un algorisme de clustering basat en la densitat, molt utilitzat quan els grups de dades no tenen una forma clara i poden contenir soroll (outliers). Encara que aquest algorisme treballa amb densitats aquest no suposa que les dades tinguin una densitat probabilística suposada.

- **Clustering jeràrquic** (*Hierarchical Clustering*): L'enfocament jeràrquic crea una estructura en forma d'arbre (dendrograma) que mostra com es formen o es divideixen els grups de dades. Pot funcionar de dues maneres principals: aglomeratiu (*bottom-up*) o divisor (top-down). L'exemple mostra com 7 clústers diferents (punts de dades) es fusionen pas a pas en funció de la distància fins a formar un únic clúster gran.



Reducció de dimensionalitat (*Dimensionality Reduction*)

Un altre tipus important d'aprenentatge no supervisat és la reducció de dimensionalitat.

L'objectiu és **reduir el nombre de característiques** (features) d'un conjunt de dades **mantenint la màxima informació rellevant possible**. Molt sovint podem pensar que quan més dades més precisos seran els nostres resultats. Però això no és així en moltes ocasions ja que moltes dades poden estar correlacionades donant problemes com el soroll (informació poc rellevant o redundant), sobreajustament (overfitting), cost computacional més alt, dificultat per visualitzar i entendre les relacions entre variables.

Per tant, utilitzem la reducció de la dimensionalitat per:

- Simplificar les dades
- Facilitar la visualització
- Millorar el rendiment d'altres algorismes de machine learning

Exemple:

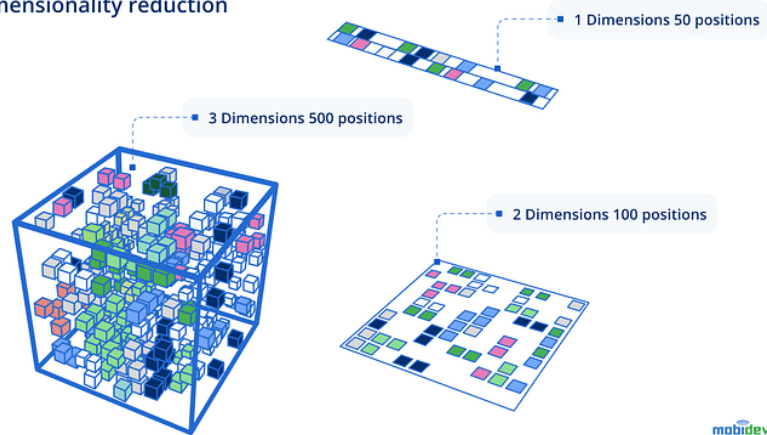
Suposem que tenim un conjunt de dades amb moltes característiques altament correlacionades. Les tècniques de reducció de dimensionalitat poden identificar els components principals que concentren la major part de la variància de les dades, i així reduir el nombre de característiques necessàries.

Per exemple:

- Característiques originals: nombre d'unitats venudes i nombre d'unitats retornades
- Nova característica: unitats netes venudes = vendes – devolucions

Això redueix dues característiques a una sense perdre informació essencial. Aquest procés també es coneix com **extracció de característiques** (*feature extraction*).

Dimensionality reduction



Origen: peerdh.com

Casos d'ús

La tècnica de reducció de dimensionalitat pot aplicar-se durant la fase de preparació de dades en projectes d'aprenentatge supervisat.

L'objectiu és eliminar dades redundants o irrelevantes, mantenint només aquelles característiques més importants per al projecte.

Exemple pràctic: Imagina que treballes en un hotel i vols predir la demanda de diferents tipus d'habitacions. Tens un gran conjunt de dades amb:

- Informació demogràfica dels clients
- Nombre de vegades que cada client ha reservat un tipus concret d'habitació l'últim any

Analitzant les dades, observes:

- Tots els clients són dels EUA → aquesta variable té variància zero i es pot eliminar.
- L'esmorzar amb servei d'habitació està inclòs en tots els tipus d'habitació → no aporta informació útil per predir la demanda.
- Les característiques "edat" i "data de naixement" són duplicades (una es pot calcular a partir de l'altra) → es poden fusionar.

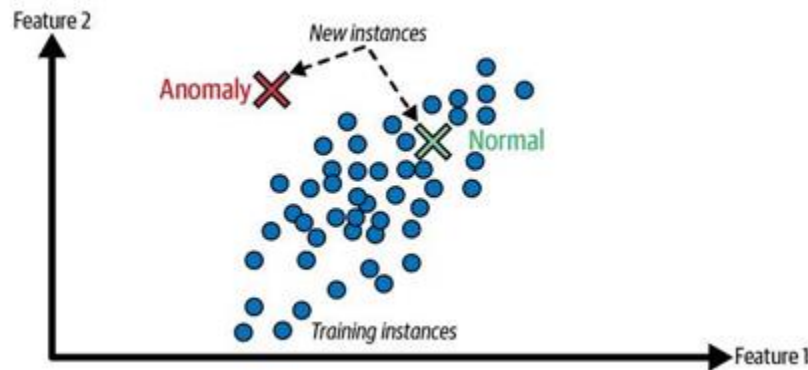
Així, redueixes la dimensionalitat del conjunt de dades, fent-lo més petit, net i rellevant per a l'anàlisi.

Tècniques habituals:

- PCA (Principal Component Analysis)
- Factor Analysis
- t-SNE (t-distributed Stochastic Neighbor Embedding)

Detecció d'anomalies

La detecció d'anomalies és una aplicació molt rellevant de l'aprenentatge no supervisat. Aquests models estan dissenyats per identificar punts de dades inusuals o inesperats que es desvien significativament de la majoria del conjunt de dades. Això és el que coneixem habitualment o anomenem valors atípics (*outliers*).



Models utilitzats:

- SVM d'una sola classe (One-class SVM)
- Bosc d'aïllament (Isolation Forest)

Aprenentatge per regles d'associació (*Association Rule Mining*)

Per últim una altra tasca que utilitza models no supervisats és l'aprenentatge de regles d'associació. L'objectiu és explorar quantitats enormes de dades i descobrir relacions interessants entre les diferents característiques.

Per exemple en un anàlisi de dades de vendes de supermercat podem trobar que la gent que compra salsa barbacoa i patates fregides també tendeix a comprar bistecs. Per tant, potser ens convingui col·locar aquests productes a prop.

Alguns models més comuns:

- Algorisme Apriori
- Algorisme FP-growth (*Frequent Pattern Growth*)

Aquests mètodes identifiquen conjunts d'articles que apareixen junts amb freqüència i calculen mesures com:

- Suport (support): freqüència amb què apareix la combinació.
- Confiança (confidence): probabilitat que un article sigui comprat quan s'ha comprat l'altre.
- Elevació (lift): grau en què la presència d'un article incrementa la probabilitat de compra d'un

altre.

Aprentatge per reforç (Reinforcement Learning)

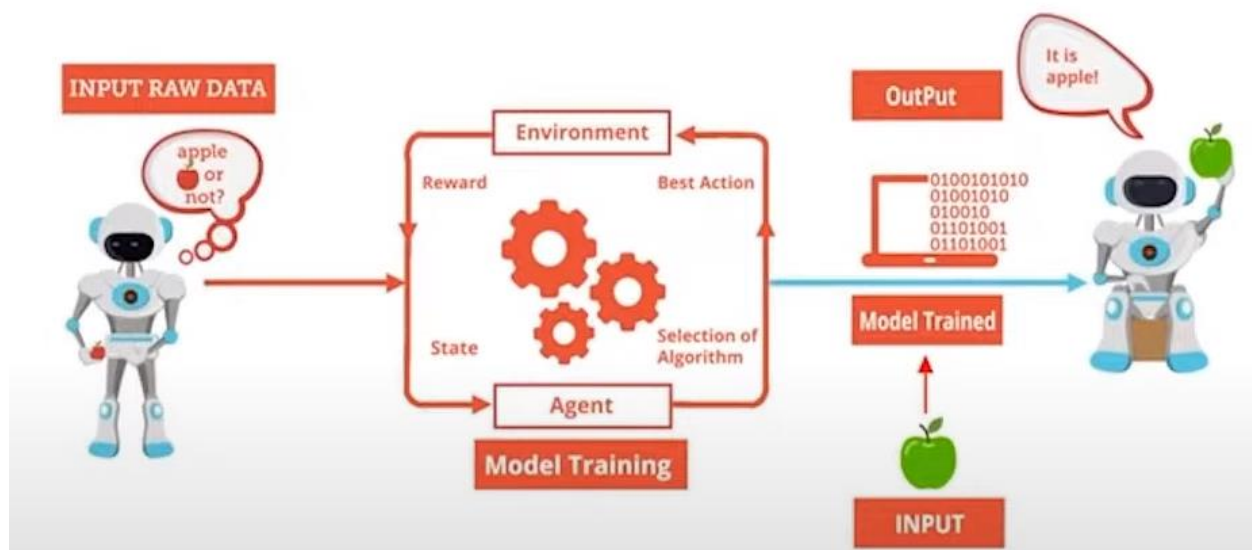
L'algorisme d'aprenentatge de reforç és força diferent dels altres que s'han comentat. En lloc d'aprendre a partir d'un conjunt de dades fix, un agent de reforç aprèn interactuant de manera iterativa constant amb el seu entorn i basada en "prova i error".

Es refereix a la interacció entre l'entorn i l'agent d'aprenentatge. L'agent d'aprenentatge es basa en l'exploració i l'explotació.

Hi ha diversos conceptes clau en l'aprenentatge per reforç:

- **Agent:** L'aprenent que pren decisions.
- **Accions:** Les opcions que l'agent pot prendre dins de l'entorn.
- **Estat:** La situació actual de l'entorn.
- **Recompenses:** Retroacció positiva que rep l'agent per dur a terme accions desitjables.
- **Penalitzacions:** Retroacció negativa que rep l'agent per dur a terme accions indesitjables.
- **Entorn:** El món amb el qual l'agent interactua.

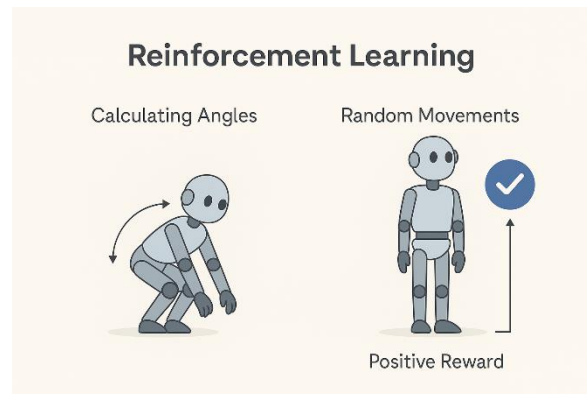
L'agent observa l'estat actual de l'entorn i després pren una acció. Aquesta acció provoca que l'entorn transiti cap a un nou estat, i l'agent rep una recompensa o una penalització segons el resultat de la seva acció. L'objectiu de l'agent és aprendre una política, és a dir, una estratègia que indiqui quina és la millor acció a prendre en qualsevol estat per tal de maximitzar les recompenses acumulades al llarg del temps.



Origen: Edureka

Un exemple podria ser el següent. Volem construir un robot capaç d'aixecar-se de terra. Tindriem 2

opcions o programar el robots de tal manera que calculem els angles i la força de cada articulació/motors tenein en compte el centre de gravetat perquè no caigui, etc... o bé podríem dir-li al robot que realitzés moviments aleatoris de tal manera que si el moviment és més proper a l'objectiu (estar dret) se li doni una recompensa i així successivament fins que el robot trobi la seqüència de moviments correctes per aixcar-se.



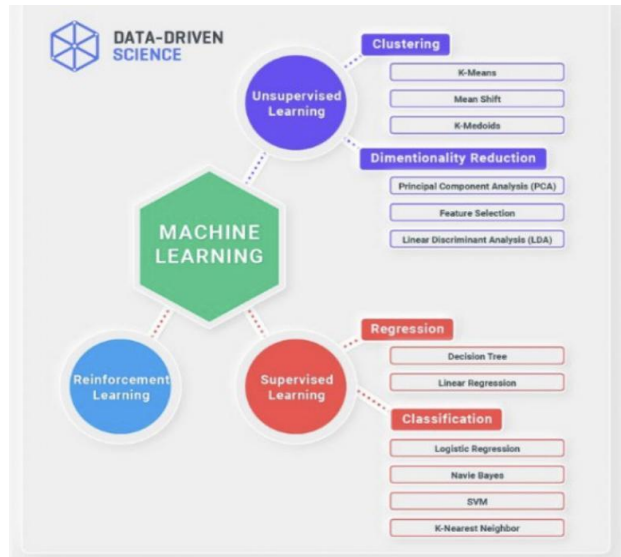
Autor: ChatGPT

Un clar exemple el tenim en **AlphaGo**, el sistema de DeepMind (filial de Google) que va derrotar el campió mundial Lee Sedol en el joc de Go l'any 2017, fet que va tenir un gran ressò mundial. El model va ser entrenat analitzant milions de partides de Go jugades per humans per aprendre les regles bàsiques i les estratègies. Encara més important, després va jugar innumbrables partides contra ell mateix, perfeccionant la seva política sobre quina és la millor jugada en cada situació mitjançant assaig i error, i maximitzant la seva "recompensa" (guanyar la partida).

El curiós del cas és que a AlphaGo no se li van ensenyar les regles del joc GO. [Vídeo1 \(ENG\)](#), [Vídeo2\(ES\)](#).
[Documental AlphaGo \(2017\) amb subtítols en castellà.](#)

Un altre clar exemple és [AWS Deep Racer](#).

Principals algorismes de ML



[Documental sobre Machine Learning i l'entrenament humà](#)

Reptes del Machine Learning

El fet que hàgim de seleccionar un model i entrenar-lo amb dades, ens podem trobar amb dos problemes. En seleccionar un "mal model" o tenir "males dades"

Dades de poca qualitat

Una de les **primeres coses que hem de revisar** abans d'entrenar un model és si les dades d'entrenament contenen **errors, valors atípics o soroll**

Tot i que pugui semblar una tasca menor, val molt la pena dedicar temps a netejar les dades abans de qualsevol entrenament. Aquesta etapa rep el nom de **Data Cleaning** i dins el món *Data Science* és una de les activitats que consumeix més temps i esforç.

Valors atípics (*outliers*)

Són dades que estan molt allunyades del comportament normal.

Què podem fer-hi?

- Eliminar les instàncies que contenen aquests valors extrems.
- Corregir-los si sabem que són deguts a un error.
- O, en alguns casos, mantenir-los si tenen sentit (per exemple: un client amb una despesa molt alta però real).

Valors que falten (*missing values*)

És molt habitual trobar dades incompletes. Per exemple, algunes instàncies poden no tenir registrat un atribut.

Opcions per tractar-ho:

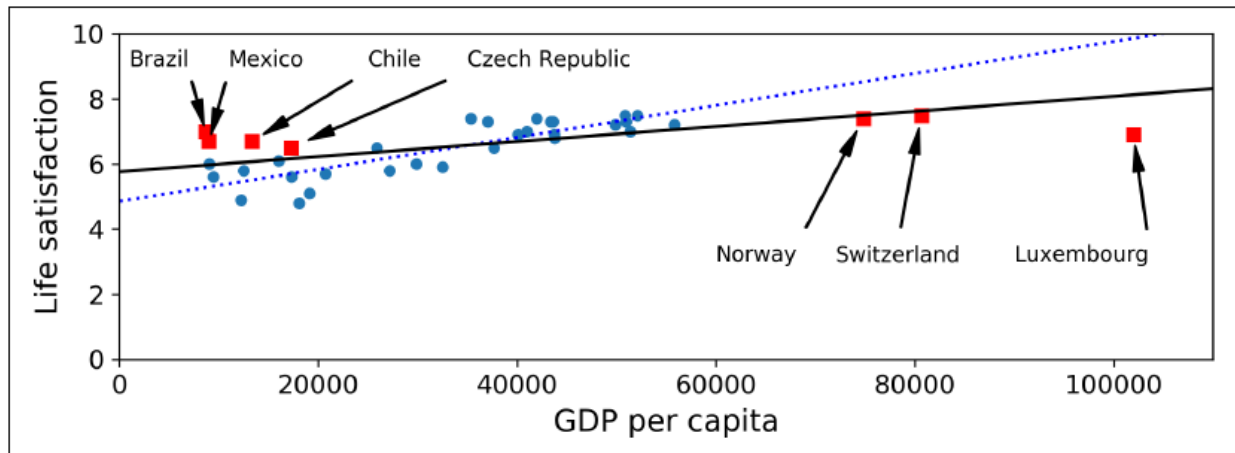
- Descartar l'atribut si té massa valors buits (i no és rellevant).
- Eliminar només les files amb valors que falten.
- Omplir els valors que falten amb: La mitjana (si són números), El valor més comú (si és categòric). Una estimació basada en altres valors similars.
- Entrenar dos models: un amb l'atribut complet i un altre sense, i comparar quin funciona millor.

Dades d'entrenament no representatives

Per generalitzar és bàsic que les dades d'entrenament siguin representatives dels nous casos als quals volem generalitzar.

Per exemple, el conjunt de països que hem utilitzat per entrenar el model no era perfectament

representatiu, **faltaven alguns països (quadrats vermells)**. Es veu que si s'haguessin afegit aquests països la recta de regressió seria diferent.



Dades desbalancejades

En problemes de **classificació**, un **conjunt de dades desbalancejat** és aquell en què hi ha molts més exemples d'una classe que d'una altra. Això pot fer que el model aprengui a ignorar les classes minoritàries i no sigui capaç de detectar-les correctament.



El model pot tenir una **precisió aparentment alta**, però **no detectar** mai la **classe** que realment ens interessa.

Per exemple si volem detectar una certa malaltia rara a una població i aquesta malaltia és molt minoritària 1% de la població. Si agafem una representació de la població per l'entrenament pot ser que no detectem la malaltia perquè aquell subconjunt no hi ha ningú amb aquella malaltia.

Per arreglar aquest tipus de problemes podem:

- **Down-sampling:** Reduir el número d'exemples de la classe majoritària fins tenir una població balancejada i poder entrenar el model.

Per exemple Si tenim 10.000 exemples de "sans" i només 100 de "malalts", seleccionem aleatòriament 100 "sans" i els combinem amb els 100 "malalts" → 200 exemples balancejats.

Un dels invonvenients que podem tenir és que perdrem molta informació útil de la classe majoritària.

- **Over-sampling:** Augmentar el número d'exemples de la classe minoritària duplicant dades o creant-ne de noves sintèticament amb tècniques com SMOTE (*Synthetic Minority Over-sampling Technique*).

Per exemple tenim 100 "malalts" i 10.000 "sans". Generem 9.900 exemples nous de "malalts" (amb SMOTE o duplicació) per tenir 10.000 de cada

El problema que ens podem trobar utilitzant aquest tècnica és caure en el sobreajustament (*overfitting*)

- **Matriu de cost:** Penalitzar més els errors comesos al classificar erròniament una classe minoritària durant l'entrenament.

Si el model confon un "malalt" amb "sà", li assignem una penalització més alta que si confon un "sà" amb "malalt". Això fa que li doni més importància a classificar bé la classe minoritària.

Aquesta tècnica és útil quan no volem tocar les dades, però requereix d'ajustar bé els pesos de penalització.

Quan treballem amb dades reals, no sempre són equilibrades. Si ignorem aquest fet, podem construir models que semblin bons però que no serveixin en la pràctica. Per això, és fonamental:

- Analitzar la distribució de les classes
- Aplicar tècniques com *down-sampling*, *over-sampling* o matrius de costos
- Avaluar bé amb mètriques adequades (no només *accuracy*!)

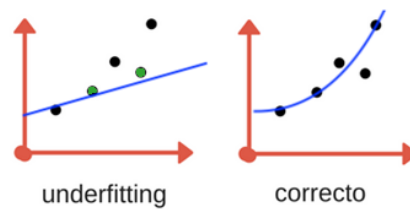
Dades d'entrenament insuficients (*Underfitting*)

Si volem que un nen petit aprengui què és una poma i la distingeixi de les altres fruites hem de presentar-li una poma i dir-li que això és una poma. El cervell del nen assimilarà la forma i color de la poma per distingir-les de les altres fruites. Potser hem de repetir aquest procés algunes vegades per ensenyar-li que hi ha pomes d'altres colors (vermelles, grogues, verdes,...) però amb pocs exemples serà suficient.

Aquest no és el cas del machine learning. La majoria d'algoritmes requereixen de moltes dades perquè funcionin bé.

La quantitat de dades de l'entrenament han de ser suficients. Si tenim poques dades, és difícil trobar un patró generalista.

En el gràfic veiem que les dades d'entrenament només han estat dues (les de color verd) amb aquestes dades el model ens marca una recta que no té res a veure amb el que hauria de ser.



Un altre exemple és si en el nostre model utilitzem només una raça de gossos per entrenar el model i llavors volem que reconegui a altres 10 races diferents l'algorisme no serà capaç de donar-nos un bon resultat.

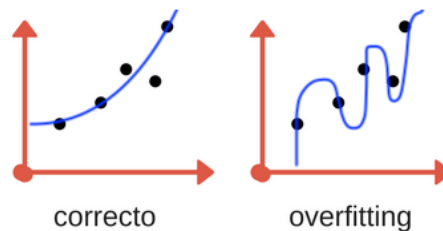
Overfitting Training Data (sobreajustament)

L'overfitting es produeix quan un model aprèn massa bé les dades d'entrenament, fins al punt que memoriza els exemples en lloc d'entendre'n els patrons generals.

Això fa que funcioni molt bé amb les dades amb què ha estat entrenat, però fracassi quan es troba amb dades noves, encara que siguin correctes o similars.

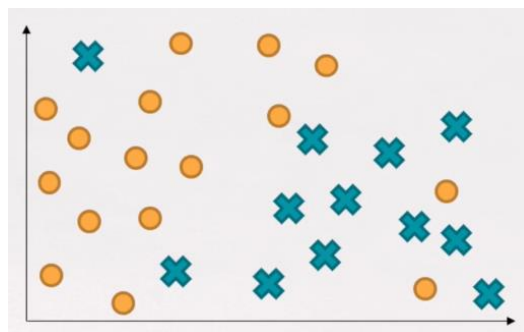
En la següent gràfica veiem que el que volem modelar és una paràbola, però el nostre model dona invàlides dades vàlides.

Resumint, tenim overfitting quan el nostre model només pot produir resultats singulars i amb la impossibilitat de comprendre noves dades d'entrada.

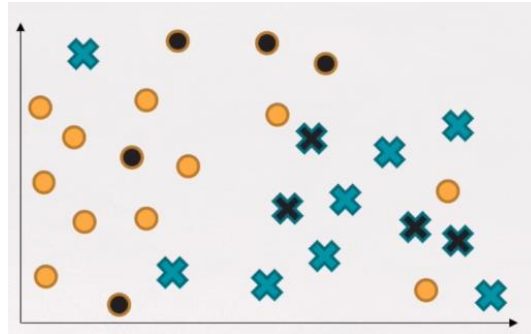


Exemple

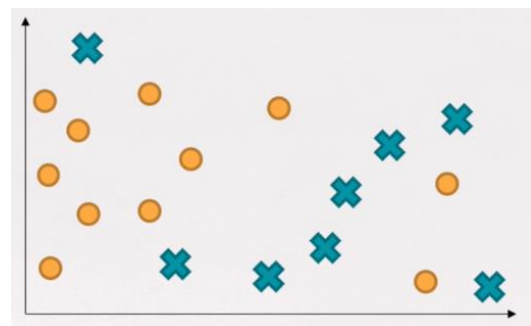
Donades aquestes dades volem aplicar un model de classificació perquè ens classifiqui cercles i creus.



Tal com hem dit hem d'agafar un subconjunt per test/validació i un altre per l'entrenament. En aquest cas marquem amb color negre les dades de test.

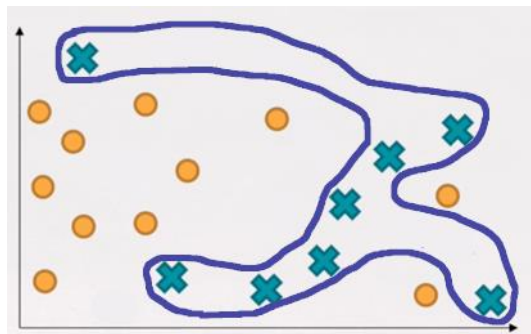


Per tant les dades d'entrenament són aquestes:

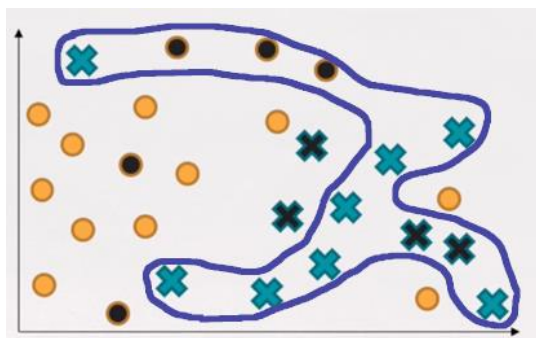


Si sobreentrenem (*overfitting*), intentem d'agafar tots els valors com a bons. En aquest exemple tenim un valor atípic a dalt a l'esquerra (una creu) i dos a baix a la dreta (dos cercles)

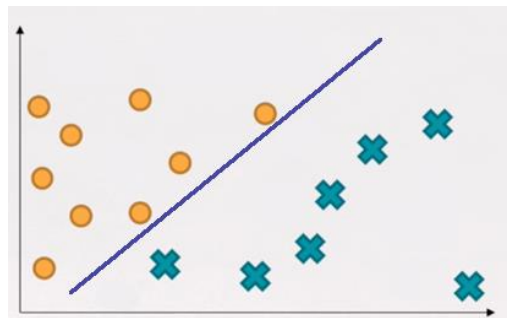
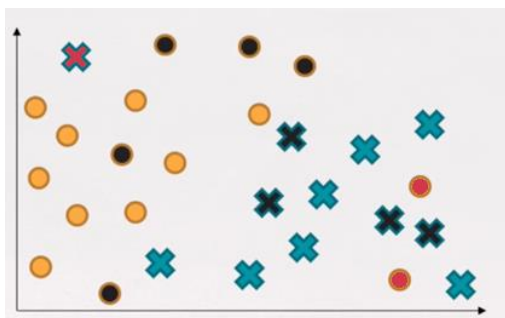
El resultat del model amb overfitting seria aquest.



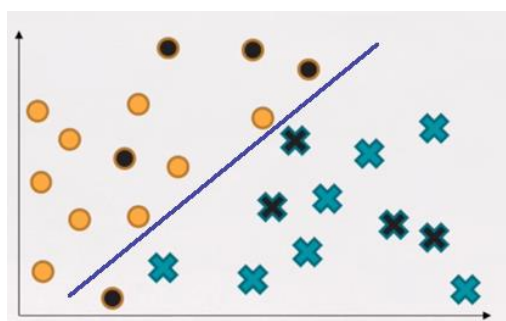
Veiem que el model amb les dades de test les classifica malament. De les 9 dades de test 5 les classifica malament això és un 55% d'error.



En canvi si el model l'entrenem sense les dades atípiques.



Amb aquest simple canvi veiem que només classifica malament 1 valor de 9, un 11% d'error. Però a simple vista veiem que el model generalitza molt millor.



Validació

El gran repte que tenim quan entrenem un model de Machine Learning és veure si funciona bé o no. La única forma de saber si el nostre model generalitza bé o no és provant nous valors.

Podríem pensar que la millor manera de saber si el model funciona és posar-lo en producció i monitoritzar-lo per a veure què passa. Això podria funcionar, però és arriscat. Si el model no funciona bé, els usuaris rebran males recomanacions o decisions incorrectes i això pot fer que perdin la confiança en el sistema i costi molt de recuperar-la.

Una opció millor, tal i com s'ha comentant anteriorment, és **dividir** les dades que tenim en **dos conjunts**: el **conjunt d'entrenament** i el **conjunt de test**. La idea és entrenar al model amb el subconjunt de les dades que hem anomenat d'entrenament i validar el model amb dades del subconjunt que hem anomenat test. D'aquesta manera podrem provar el model amb dades que no ha vist mai i així poder avaluar el model. Si l'error d'entrenament és baix amb les dades d'entrenament i alt amb dades de test significarà que tenim el nostre model sobreentrenat i estem caient en *overfitting*.



Com també s'ha dit anteriorment és habitual utilitzar el 80% de les dades per l'entrenament i reserva el 20% per el test.

El conjunt de dades de test haurà de tenir diversitat i una quantitat suficient per poder comprovar els resultats.

Cas d'ús – Projecte de Machine Learning

Tal i com s'ha dit en el punt “**Com treballa el Machine Learning?**” Els passos a realitzar per qualsevol projecte d'aprenentatge automàtic són:

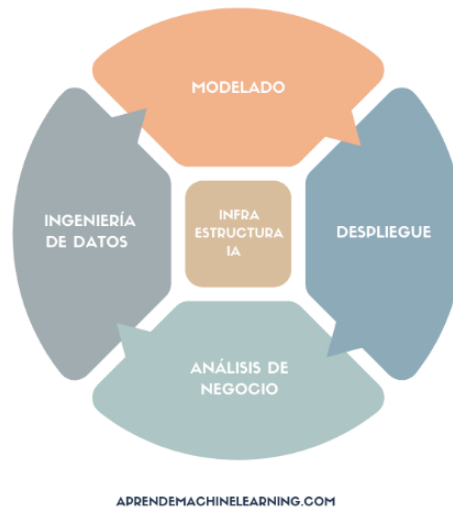
1. Mirar el problema i tenir una visió general.
2. Obtenir les dades.
3. Descobrir i visualitzar les dades per obtenir informació (Exploració de dades – EDA)
4. Preparar les dades per als algorismes d'aprenentatge automàtic.
5. Seleccionar un model i entrenar-lo.
6. Afinar el model (avaluar/validació del model).
7. Presentar la solució.
8. Iniciar, supervisar i mantenir el sistema.



Les fases 2, 3 i 4 les podem aglutinar en una sola fase anomenada **enginyeria de dades**.

Les fases 5, 6 i 7 les podem aglutinar en una sola fase anomenada **modelat**.

CICLO DE VIDA DE UN PROYECTO MACHINE LEARNING



1) Infraestructura d'IA

La infraestructura és una “etapa” transversal i que afecta a totes les altres.

Una decisió important és per exemple si som nosaltres mateixos que muntarem i mantindrem la infraestructura o si utilitzarem serveis externs

2) Anàlisi de Negoci

En aquesta etapa es defineixen els objectius que es volen aconseguir mitjançant l'IA. Es creen o suggereixen mètriques que s'utilitzaran per avaluar els resultats.

El primer que cal saber és quin és l'objectiu empresarial del que volem. Segur que l'objectiu no és crear un model sinó de quina manera l'empresa es vol beneficiar d'aquest model.

3) Enginyeria de Dades

Aquesta etapa inclou la recollida de dades de diferents fonts (bases de dades, fitxers semi-estructurades, dades no estructurades, APIs públiques,...) i els seu tractament, preprocessat i futur manteniment.

Tenim barreres a afrontar que a part de les diferents fonts són el volum de dades que hem de processar.

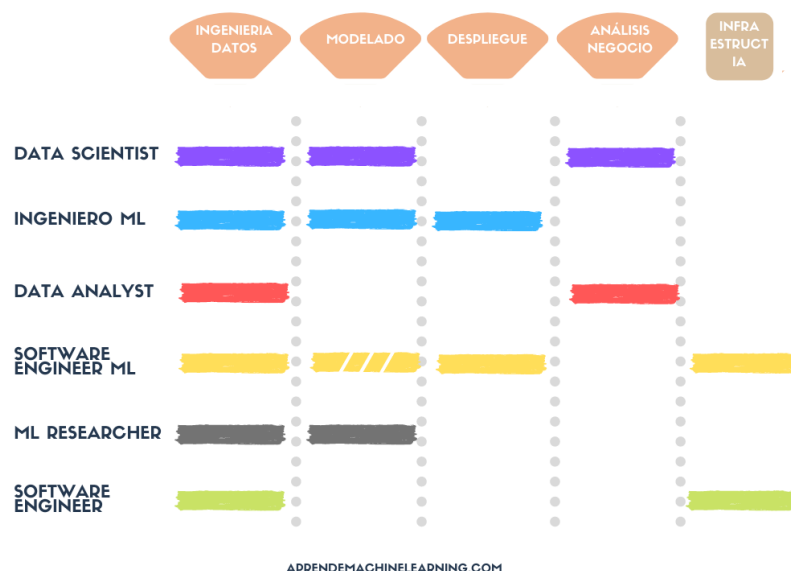
4) Modelat

Aquesta etapa és a on es fa la “màgia” i utilitzarem totes les habilitats en Data Science. Seleccionarem els models i els avaluarem.

5) Implementació i desplegament

Durant l'etapa d'implementació i desplegament formalitzarem el codi prototipus en un pipeline robust i consistent.

Rols en els projectes de Machine Learning



DATA SCIENTIST

El científic de dades pot desenvolupar les etapes d'anàlisi de negoci, enginyeria de dades i modelat. Ha de tenir fonaments científics sòlids, habilitats de comunicació per poder transmetre els resultats als altres membres de l'equip o usuaris del negoci.

ENGINEER ML

Pot desenvolupar les etapes d'enginyeria de dades, modelat i desplegament. En alguns casos també pot col·laborar amb l'anàlisi de negoci i en la infraestructura.

Té les habilitats en enginyeria, però també en ciències. Les seves competències comunicatives poden dependre de les necessitats de l'equip.

Aquesta persona ha d'estar actualitzat en les últimes tendències en algorismes i informes amb el Machine Learning.

DATA ANALYST

Preparat per les primeres fases: anàlisi de negoci i enginyeria de dades. En general tenen un gran coneixement d'SQL i manipulació de bases de dades així com analítica avançada de negoci visualització i *reporting*. També utilitza eines de *Business Intelligence* com Power BI, Qlik o Tableau.

També ha de ser un rol amb bones habilitats comunicatives i se li exigeix menys capacitat d'algorísmica o de programació.

ENGINEER DE SOFTWARE DE ML

Aquestes persones podran desenvolupar tasques en les etapes d'enginyeria de dades, modelats, desplegament i infraestructura.

Aquest rol pot cobrir molts tipus de tasques diferents i seria convenient en etapes embrionàries dels equips.

ENGINEYER DE SOFTWARE

Aquest rol (actualment mot lligat a “devops”) pot ocupar-se de les etapes d'enginyeria de dades i infraestructura.

Han de demostrar gran habilitats en els àmbits de la programació i amb diverses eines o plataformes especialitzades.

INVESTIGADOR ML

Aquest rol estarà en les etapes d'enginyeria de dades i del modelat. Desenvolupen el seu potencial en un entorn d'investigació per buscar i descobrir patrons de dades. Han de tenir excel·lents habilitats i coneixement científic.

Poden especialitzar-se en Deep Learning, NLP, visió artificial, etc...

En el següent enllaç/article podem trobar diferents eines utilitzades en el món del DataScience:

<https://dev.to/minchulkim87/my-data-science-tech-stack-2020-1poa>

Fase d'Exploració de dades (EDA)

L'exploració de dades o també coneguda com a anàlisi exploratòria de dades (**EDA** - **Exploratory Data Analysis**) és una fase molt important o essencial destinada a descobrir i visualitzar les dades per obtenir-ne informació. Aquesta fase la realitzarem un cop tenim clar de quines dades disposem i com s'obtindran.



Ens ajuda a identificar qualsevol patró ocult a les dades, la possible correlació entre diferents columnes i/o l'anàlisi de les propietats de les dades, però hem de tenir clar que aquesta fase no té sentit **si no tenim quin és l'objectiu que volem aconseguir** a partir de les dades i per tant abans hem d'haver passat per una fase de visió general a on s'indiqui quin és l'objectiu empresarial que volem. Per exemple, **“Volem predir les vendes a 30 dies”**, o **“Classificar casos malignes/benignes d'una malaltia”**, **“Volem fer un pronòstic de fidelització de clients/abonaments”**, **“Volem detectar casos de frau”**.

En general l'EDA ocupa al voltant del 30% del temps total del projecte perquè necessitem escriure molt de codi per crear diferents tipus de visualitzacions i analitzar-les.

Aquesta fase no la farà una sola persona o un sol rol de l'equip. Sinó que serà un treball conjunt de l'equip a on també s'haurà d'utilitzar l'ajuda dels experts de les dades que s'estan tractant. Per exemple els metges que ens estan proporcionant les dades mèdiques a tractar.

Algunes de les preguntes que ens podem fer són:

- De quants registres disposem? Són massa pocs? Són molts i no tenim capacitat (CPU+RAM) suficient per processar-los?
- Tenim totes les files completes o tenim camps amb valors nuls o en blanc?
- En cas que hi hagi massa nuls: la resta d'informació queda inútil?
- Quines dades són discretes i quines contínues?
- Moltes vegades serveix obtenir el tipus de dades: text, int, double, float
- Si és un problema de tipus supervisat:
 - Quina és la columna de “sortida”? binària, multiclasse?
 - Està balancejat el conjunt sortida?
 - Quins semblen ser features importants? Quins podem descartar?
- Segueixen alguna distribució?

- Hi ha correlació entre features (característiques)?
- En problemes de NLP és freqüent que hi hagi categories repetides o mal tipejades, o amb majúscules/minúscules, singular i plural, per exemple “Advocat” i “Advocades”, “advocat” pertanyerien tots a un mateix conjunt.
- Estem davant d’un problema dependent del temps? És a dir, un TimeSeries.
- Si fos un problema de Visió Artificial: Tenim prou mostres de cada classe i varietat, per poder fer generalitzar un model de Machine Learning?
- Quins són els *Outliers*? (unes poques dades aïllades que difereixen dràsticament de la resta i “contaminen” o desvien les distribucions).
 - Podem eliminar-los? és important conservar-los?
 - Són errors de càrrega o són reals?
- Tenim possible biaix de dades / dades desbalancejades? (per exemple perjudicar classes minoritàries per no incloure-les i que el model de ML discrimini)
- L’exploració de dades dependrà del problema o el projecte que estiguem tractant, però podem tenir un checklist genèric per quan ens enfrontem en un problema general
 - a) Resums d’estadístiques descriptives: mitjana, median, desviació estàndard, mínim, màxim.
 - b) Distribucions de freqüència: histogrames
 - c) Percentils i Boxplots
 - d) Correlacions i dispersió

El primer pas que hem de fer és familiaritzar-nos amb les dades que tenim: veure quines columnes tenim, veure les 5 primeres i últimes files, etc..

Un altra cosa a fer és comprovar els diferents tipus de dades que tenim per cada columna.

Aquí comprovem els tipus de dades perquè de vegades el preu del cotxe s’emmagatzema com una cadena de text. En aquest cas hem de convertir aquesta cadena a dades senceres o decimals per poder-les representar en un gràfic.

Estadística descriptiva

Alguns dels estadístics com per exemple comptar el número d’elements, la mitjana, la mediana, la desviació típica, el valor màxim, el valor mínim ens poden ajudar a introduir-nos a les dades que tenim i a tenir-ne un resum o detectar errors. Per exemple si la mitjana de l’edat és de 35 anys, però el màxim és 230, probablement hi ha un error en algun valor.

Preparació de les dades

Quan treballem amb dades reals és molt habitual trobar que cada variable/característica té unitats, rangs i magnituds diferents. Després de realitzar una exploració de les dades el que cal és realitzar transformacions concretes per imputar valors perduts, eliminar *outliers*, codificar variables categòriques.

Per què es important transformar les dades?

Quan barregem variables/característiques en escales molt diferents (€, anys, m²...), els algorismes “donen més veu” a magnituds que numèricament són més grans, tot i que no siguin més importants.

Des d'un punt de vista matemàtic aquestes diferències poden crear problemes d'escala. Molts algorismes de ML treballen amb distàncies o que ajusten pesos mitjançant gradients assumint que totes les característiques estan més o menys dins el mateix rang numèric.

Per exemple, si volem predir si una persona comprarà o no un producte en funció de l'edat i els seus ingressos ens trobarem que el rang d'ingressos són valors que van de 0 a 100.000€ i en canvi l'edat de 18 a 90.

Les distribucions molt asimètriques i *outliers* poden alentir o desestabilitzar l'entrenament.

Molts dels models treballen amb dades numèriques. És per aquest motiu que cal convertir les dades categòriques a numèriques.

EXEMPLES DE ELIMINAR OUTLIERS/ NULLS.

Transformacions lineals a variables numèriques (no modifiquen distribució)

Els escalats lineals ajuden les dades a una nova escala, però **no modifiquen la forma de la distribució**. Això vol dir que si una variable era asimètrica o tenia *outliers*, continuarà tenint-los, simplement els valors quedaran dins d'un altre rang.

Min-Max Scaler

El *Min-Max Scaler* és el mètode més senzill i intuïtiu: transforma cada valor de x perquè caigui dins un interval concret. Habitualment entre **0 i 1**

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Això vol dir que el valor més petit de la variables es convertirà en 0, i el més gran en 1 . La resta de valors quedaran proporcionalment entre mig.

Aquest tipus de transformació és **molt sensible als outliers**: Si apreix un valor molt gran o molt petit, pot distorsionar l'escala de tota la resta de dades.

S'utilitza en KNN, K-Means, xarxes neuronals o qualsevol mètode basat en distàncies, perquè aquests algorismes mesuren "com lluny estan" els punts en l'espai multidimensional, i si la variable està en una escala molt gran pot dominar respecte les altres.

```
from sklearn.preprocessing import MinMaxScaler
import pandas as pd

data = pd.DataFrame({'edat': [18, 25, 30, 45, 80]})
scaler = MinMaxScaler()
data['edat_escalada'] = scaler.fit_transform(data[['edat']])
print(data)
```

Standard Scaler (Z-score)

Aquest és, probablement, el més popular. El *Standard Scaler* centra les dades de manera que la **mitjana sigui 0 i la desviació estàndard sigui 1**.

Matemàticament, es calcula amb:

$$z = \frac{x - \mu}{\sigma}$$

On μ és la mitjana i σ la desviació estàndard.

Aquest tipus d'escalat és especialment útil en models que **assumeixen una distribució normal** o que funcionen millor amb dades centrades, com ara la **Regressió Lineal**, la **Regressió Logística**, els **SVM** o les **Xarxes Neuronals**.

No obstant això, és important tenir en compte que aquest escalat no elimina l'efecte dels *outliers*: aquests valors extrems continuaran influint en la mitjana i la desviació.

```
from sklearn.preprocessing import StandardScaler
import numpy as np

x = np.array([[10], [12], [13], [15], [100]]) # l'últim és un outlier
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)
print(x_scaled)
```

Normalizer

El *Normalizer* és una mica diferent. No escala cada **columna**, sinó cada **fila** (cada mostra). És a dir, converteix cada vector de característiques en un vector de **longitud 1**, de manera que totes les mostres tinguin la mateixa norma.

Això és molt útil en models on volem comparar vectors segons la seva **direcció** i no la seva magnitud, com passa en el càlcul de **similitud cosinus** o en vectors de text (TF-IDF, embeddings...).

```
from sklearn.preprocessing import Normalizer
data = pd.DataFrame({
    "feature1": [1, 2, 3],
    "feature2": [2, 4, 6]
})
scaler = Normalizer()
print(scaler.fit_transform(data))
```

```
skew
Normal    0.02
Dreta     1.98
Esquerra  -1.92
```

Transformacions no lineals a variables numèriques (si modifiquen distribució)

Hi ha casos en què no n'hi ha prou amb canviar l'escala: les dades poden estar molt **asimètriques**, tenir una **cua llarga** o una **variància que depèn del valor**.

En aquests casos necessitem transformacions que **modifiquin la forma de la distribució** per fer-la més “normal”.

Logarithmic Transform

Quan tenim variables amb valors positius molt grans i asimetria forta (com els ingressos o les vendes), una transformació logarítmica pot ajudar molt.

Redueix la separació entre valors grans i petits, fent que la distribució sigui més propera a una campana de Gauss. El `np.log1p()` aplica el logaritme natural però sense problemes amb el zero.

```
import numpy as np
data = pd.DataFrame({'ingressos': [100, 500, 1000, 10000, 100000]})
data['log_ingressos'] = np.log1p(data['ingressos'])
print(data)
```

PowerTransformer (Yeo-Johnson / Box-Cox)

El *PowerTransformer* és una versió més flexible que busca transformar automàticament les dades perquè siguin el més “gaussianes” possibles.

Internament pot utilitzar dos mètodes:

- **Box-Cox**, que només serveix per valors positius.
- **Yeo-Johnson**, que també funciona amb valors negatius o zero.

Aquest tipus de transformació és molt potent per estabilitzar la variància i millorar el comportament de models **lineals o regressions**.


```
from sklearn.preprocessing import PowerTransformer
scaler = PowerTransformer(method='yeo-johnson')
data = pd.DataFrame({'ingressos': [100, 500, 1000, 10000, 100000]})
data['transformada'] = scaler.fit_transform(data[['ingressos']])
```

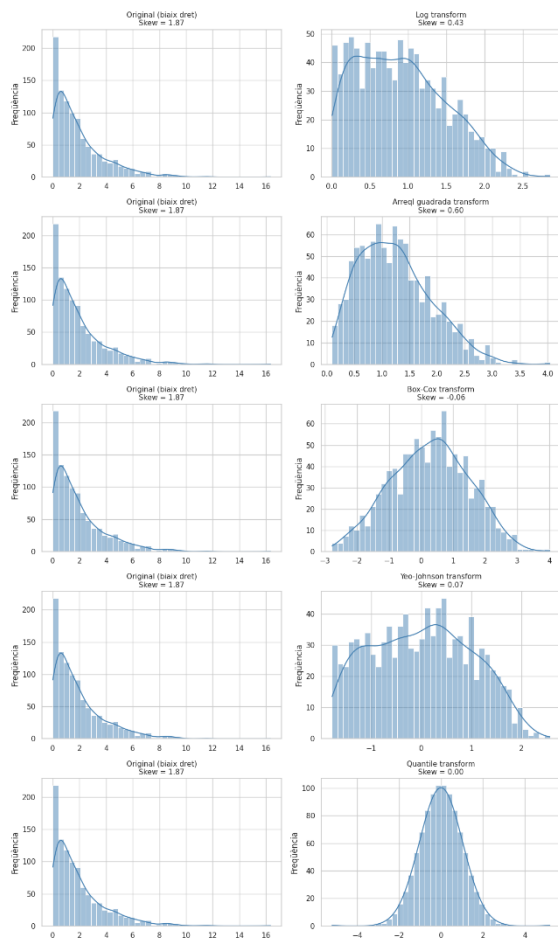
QuantileTransformer

El *QuantileTransformer* va un pas més enllà. Converteix els valors originals segons la seva **posició en la distribució (quantil)** i després els mapeja a una **distribució uniforme o normal**.

Això fa que la nova variable tingui una forma molt més regular i robusta als *outliers*.

Això pot millorar notablement models sensibles a la distribució, com **regressions o SVM**, però a canvi **perd la interpretabilitat** (ja no sabrem què significa un “3.5” després de la transformació).

```
from sklearn.preprocessing import QuantileTransformer
qt = QuantileTransformer(output_distribution='normal')
data = pd.DataFrame({'ingressos': [100, 500, 1000, 10000, 100000]})
data['qt_ingressos'] = qt.fit_transform(data[['ingressos']])
```



En els següents histogrames podem veure la diferència que hi ha de cada transformador a partir de la mateix dataset inicial.

També hi ha representats els valors d'assimetria (skeness) de cada funció per veure'n el resultat numèricament.

Recorda que l' skew (també conegut com asimetria o skewness) és un valor numèric agregat que mesura el grau de simetria d'una distribució de dades respecte a la seva mitjana.

En altres paraules:

El skew ens indica si una variable té més valors concentrats a l'esquerra o a la dreta del seu centre, i quant s'allunya de la forma perfecta d'una distribució

normal.

Quan apliquem les transformacions?

Quan transformem les dades (per exemple, fent un escalat amb *StandardScaler*), només s'han d'aplicar a les dades d'entrenament (*train*).

Després, apliquem exactament la mateixa transformació al conjunt de test, però sense tornar a "aprendre" res d'aquestes dades noves.

Si fem servir informació del test per calcular mitjanes, desviacions o escales, el nostre model ha vist informació "del futur" i ens estem fent trampes al solitari → Això és el que es coneix com a una fuga de dades (**data leakage**).

Això fa que el model tingui resultats artificialment bons i que quan el provem amb dades reals, rendeixi pitjor del que esperàvem.

Exemple de cas incorrecte amb data leakage:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Dataset fictici
np.random.seed(0)
df = pd.DataFrame({
    "area": np.random.normal(100, 20, 10),
    "rooms": np.random.randint(2, 6, 10),
    "price": np.random.normal(200000, 50000, 10)
})

#1 ERROR: escalar abans de dividir
scaler = StandardScaler()
df_scaled = df.copy()
df_scaled[["area", "rooms"]] = scaler.fit_transform(df[["area", "rooms"]])

#2 Després dividim (tard)
X_train, X_test, y_train, y_test = train_test_split(
    df_scaled[["area", "rooms"]], df_scaled["price"], test_size=0.3, random_state=0
)
```

Exemple de cas correcta sense data leakage

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Dividim primer
X = df[["area", "rooms"]]
y = df["price"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)
```

```
# Escalem correctament
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit + transform només amb train
X_test_scaled = scaler.transform(X_test)      # transform, però sense tornar a “aprendre”
# codi
```

Exemple d’us del Pipeline de Scikit-learn

Si utilitzem el Pipeline de Scikit-learn es garanteix que totes les transformacions només s’ajustin al *train* i que el *leakage* sigui impossible.

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LinearRegression())
])

# Entrenament
pipeline.fit(X_train, y_train)

# Predicció (aplica automàticament la mateixa escala al test)
y_pred = pipeline.predict(X_test)
```

Altres fuites d’informació:

- **Imputar valors nuls abans de dividir:** Calcular la mitjana de tot el dataset influeix en la mitjana usada al train.
- **Utilitzar variables derivades de la *target*.** Exemple si estem intentant de predir el preu i tenim una característica anomenada “mitjana del preu per barri”. Aquesta dada ja conté informació de la

variable objectiu.

Resum

No tots els models necessiten escalat. Alguns (com els arbres de decisió o els boscos aleatoris) són completament invariants a l'escala de les dades.

Altres, en canvi, en depenen molt. A continuació hi ha una mica de taula orientativa, tot i que sempre cal realitzar proves.

Tipus de model	Necessitat escalat	Transformacions	Comentaris
Regressió lineal / logística	Sí ✓	StandardScaler.	Millora convergència i evita que un atribut domini els altres.
SVM / KNN / K-Means / PCA	✓ ✓ Sí	MinMaxScaler StandardScaler	Basats en distàncies
Random Forest / Gradient Boosting / XGBoost	✗ No necessari	Cap	Els arbres divideixen segons ordres, no magnituds.
Xarxes neuronals	✓ Sí	StandardScaler. Cal disminuir els valors grans.	Els valors molt grans saturen les funcions d'activació.
Models de text (TF-IDF, embeddings)	Normalització per fila	Normalizer	S'utilitza Normalizer per comparacions angulars

Conclusions:

- Escalar no és opcional: és una part fonamental del preprocessing. Una dada mal escalada pot fer que un model funcioni molt pitjor.
- Cada tipus d'escalat té el seu moment. Si tens outliers, no facis servir StandardScaler; si tens distribucions molt rares, considera PowerTransformer o QuantileTransformer.
- Els escalats lineals (com Standard, MinMax i Robust) només ajusten les unitats; les transformacions no lineals (com Power o Quantile) també modifiquen la forma de la distribució.

- Visualitzar abans i després és clau per entendre si la transformació ha estat beneficiosa.

Finalment, tot escalat s'ha d'aplicar dins d'un Pipeline i sempre després de dividir entre train i test, per evitar que la informació del test "contamini" l'entrenament (el que anomenem data leakage).

Codificació de variables categòriques

Quan treballem amb ML, la majoria d'algorismes només treballen amb valors numèriques. No poden calcular una distància entre "Barcelona" i "Girona", ni entendre què significa "nivell mitjà" o "alt".

Per això, abans d'entrenar un model, les variables categòriques (anomenades també qualitatives o discretes) s'han de codificar numèricament.

Ara bé, la manera com fem aquesta codificació pot influir molt en el resultat final del model. No és el mateix representar les categories amb un codi ordinal (0, 1, 2) que amb variables binàries independents.

Les transformacions categòriques més utilitzades són:

- One-Hot Encoding
- Ordinal Encoding
- Target Encoding

(Altres menys comunes: Hash Encoding, Frequency Encoding, etc.)

One-Hot Encoding

És la transformació més habitual

És la transformació més "segura" perquè:

- No introdueix ordres falsos: Per exemple si tenim $a=1$, $B=2$, $C=3$. El model pot interpretar que hi ha una relació d'ordre/distància entre les categories. Cosa que no és cert.
- Perquè és determinista i interpretable
- Perquè no crea dependències numèriques ocultes. Cada categoria es representa en una dimensió pròpia. La distància entre categories és sempre la mateixa. No hi ha una correlació artificial entre

columnes.

- És robust davant dades noves (OneHotEncoder(handle_unknown='ignore')

El seu funcionament és simple: crea una nova columna per a cada categoria i assigna un 1 o un 0 segons si l'observació pertany o no a aquella categoria.

Per exemple, si tenim una columna "Ciutat" amb tres categories: Barcelona, Girona i Lleida, el resultat seria:

Ciutat	BCN	GIR	LLE
Barcelona	1	0	0
Girona	0	1	0
Lleida	0	0	1

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

# Exemple: colors sense ordre natural
df = pd.DataFrame({"ciutat": ["Barcelona", "Girona", "Lleida", "Tarragona"]})

# One-Hot Encoding (segur)
ohe = OneHotEncoder(sparse_output=False, handle_unknown="ignore")
ohe_features = ohe.fit_transform(df[["ciutat"]])
df[ohe.get_feature_names_out()] = ohe_features

print(df)
```

ciutat	ciutat_Barcelona	ciutat_Girona	ciutat_Lleida	ciutat_Tarragona
0 Barcelona	1.0	0.0	0.0	0.0
1 Girona	0.0	1.0	0.0	0.0
2 Lleida	0.0	0.0	1.0	0.0
3 Tarragona	0.0	0.0	0.0	1.0

Ordinal Encoding

En alguns casos, les categories tenen un ordre natural. És quan parlem de dades categòriques ordinals. Per exemple, els nivells d'assoliment ('no parovat', 'suficient', 'notable', 'excel·lent') o les talles a la roba ('S', 'M', 'L').

En aquests casos podem representar la dada categòrica amb valors ordinals.

Ens és útil quan tenim variables amb significat jeràrquic o progressiu.

```
from sklearn.preprocessing import OrdinalEncoder

nivells = pd.DataFrame({'Educació': ['Bàsic', 'Mitjà', 'Superior', 'Mitjà']})
encoder = OrdinalEncoder(categories=[['Bàsic', 'Mitjà', 'Superior']])
nivells['Codificat'] = encoder.fit_transform(nivells[['Educació']])
print(nivells)
```

Target Encoding

Quan tenim una variable categòrica amb moltes categories diferents, com poden ser noms de municipis, codis de producte o marques, el One-Hot no és pràctic.

El *Target Encoding* ofereix una alternativa elegant: substitueix cada categoria pel **valor mitjà de la variable objectiu (target)** per a aquella categoria.

Per exemple, si estem predint si un client cancel·la una subscripció (0 = no, 1 = sí), podem substituir cada municipi pel **percentatge de clients que han cancel·lat** en aquell municipi.

Així, les categories amb comportaments semblants tindran valors semblants.

Compte que si no s'aplica correctament podem produir fuites d'informació del test (data leakage). Per això es recomana utilitzar-lo dins d'un *Pipeline*

```
from sklearn.preprocessing import TargetEncoder
import numpy as np

df = pd.DataFrame({
    'Ciutat': ['Barcelona', 'Girona', 'Lleida', 'Girona', 'Barcelona'],
```



```
'Abandonament': [1, 0, 1, 1, 0]
})
encoder = TargetEncoder()
df['Ciutat_encoded'] = encoder.fit_transform(df['Ciutat'], df['Abandonament'])
print(df)
```

Tractament de biaixos (bias) en la transformació de dades

El **biaix** és un dels grans reptes del Machine Learning modern.

Quan parlem de biaix, ens referim al fet que les dades poden reflectir **desigualtats, desequilibris o patrons injustos** presents en la realitat o en la manera com es van recollir les dades.

Per exemple:

- Si en un *dataset* de contractació laboral el 80% de candidats són homes, el model pot aprendre que “home” és sinònim de “contractat”.
- Si una variable categòrica “Ciutat” té poques mostres en ciutats petites, pot quedar **infravalorada** pel model.

Aquests biaixos **no es corregeixen només amb transformacions**, però sí que el tipus de transformació pot ajudar a **reduir-los o amplificar-los**.

a) Quan les transformacions poden ajudar

Les transformacions no lineals com QuantileTransformer o PowerTransformer poden reduir biaixos estadístics o numèrics, és a dir, desequilibris de magnitud o dispersió entre valors.

Per exemple, si una variable d'ingressos té una cua molt llarga (uns pocs molt rics i molts amb poc), un log-transform o power-transform pot reduir aquest desequilibri i evitar que el model es centri només en els casos més extrems.

També, en el cas de categories rares, fer servir agrupacions de baixa freqüència (`min_frequency` a `OneHotEncoder`) pot evitar que una categoria amb poques mostres tingui massa pes o faci el model inestable.

b) Quan les transformacions poden ajudar

El *Target Encoding*, per exemple, pot introduir biaix de fuga d'informació: si s'entrena amb tot el dataset (inclòs el test), pot donar avantatge al model perquè “sap” el target.

També, si una categoria té poques mostres, el seu valor mitjà pot ser molt sorollós i distorsionar els resultats.

Per això sempre cal:

- Aplicar transformacions dins d'un **Pipeline** després de dividir en train i test.
- Fer servir **cross-validation** quan hi hagi codificacions dependents del target.
- Revisar la **distribució de les categories** i assegurar que cap grup està infrarrepresentat

c) Altres mesures de tractament del biaix

- Rebalancejar mostres (undersampling / oversampling).
- Normalitzar pesos per classe en l'entrenament
- Analitzar mètriques d'equitat: *precision*, *recall* per grups demogràfics

Collage de fotos de muffins y chihuahuas (<https://blogs.upm.es/pasd/2023/06/12/uso-de-aprendizaje-multi-tarea-para-abordar-el-problema-del-muffin-chihuahua/>)

REFERÈNCIES

- Aurélien Géron (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media.
- <https://medium.com/@vishalgimhan/types-of-machine-learning-approaches-part-1-3-ml02-0042e443d989>
- Continguts de la docent Cristina Gómez de IES de Teis Vigo